Uniwersytet Mikołaja Kopernika w Toruniu
Wydział Matematyki i Informatyki

Jakub Grędzicki
nr albumu: 316246

Praca dyplomowa inżynierska
na kierunku Informatyka

# Budowa i działanie amatorskiego mikrosatelity w formacie CubeSat

Promotor
dr Marta Burzańska

Toruń 2025

# Spis treści

**1 Wstęp** **1**

1.1 Wprowadzenie . . . . . 1

1.2 Przegląd aktualnych rozwiązań . . . . . 2

1.3 Cel pracy . . . . . 3

1.4 Zakres pracy . . . . . 4

1.5 Struktura pracy . . . . . 5

**2 Projekt** **6**

2.1 Wprowadzenie . . . . . 6

2.2 Wymagania funkcjonalne . . . . . 6

2.3 Schemat blokowy modelu . . . . . 7

2.4 Wykorzystane narzędzia . . . . . 9

**3 Warstwa sprzętowa** **10**

3.1 Moduł zasilania . . . . . 10

3.2 Moduł sterujący . . . . . 17

3.3 Moduł czujników . . . . . 23

3.4 Obudowa . . . . . 26

**4 Software** **31**

4.1 Architektura systemu . . . . . 31

4.2 Model komunikacji . . . . . 32

4.3 Menadżer stanu systemu . . . . . 39

4.4 Menadżer zasilania . . . . . 41

4.5 System zdarzeń . . . . . 43

4.6 System telemetrii . . . . . 49

4.7 Implementacja systemu plików . . . . . 55

4.8 System czujników . . . . . 56
4.9 Zegar . . . . . 62
4.10 System lokalizacji i komunikacji GPS . . . . . 65

**5 System operatora satelity** **72**
5.1 Warstwa sprzętowa . . . . . 73
5.2 Warstwa programowa . . . . . 73

**6 Testy** **80**
6.1 Testy poleceń . . . . . 80
6.2 Testy telemetrii . . . . . 80

**7 Podsumowanie** **84**
7.1 Wnioski . . . . . 84
7.2 Możliwości rozwoju . . . . . 84

# Wstęp

## 1.1 Wprowadzenie

W dzisiejszych czasach kosmiczne projekty użytkowe, takie jak satelity meteorologiczne, obserwacyjne, systemy nawigacyjne GPS czy Galileo, systemy telefonii satelitarnej i Internetu, są powszechne w naszym codziennym życiu.

Wzrost liczby startów rakiet nośnych, rozwój współpracy agencji kosmicznych z przemysłem i nauką oraz spadek cen komponentów elektronicznych i specjalistycznych materiałów, umożliwiły mniejszym przedsiębiorstwom specjalizującym się w określonych obszarach technologii, udział w opracowaniu i przygotowaniu do wystrzelenia sztucznych satelitów.

Koszty wysyłki sprzętu w przestrzeń kosmiczną są przeliczane w dolarach na kilogram masy ładunku, określanego jako tzw. payload. Współcześnie, komercyjne satelity tworzone przez uczelnie i konsorcja, także polskie, są faktem.

Elektronika w przestrzeni kosmicznej to nie tylko sztuczne satelity, ale także systemy kontrolne rakiet, łazików czy też kombinezonów przeznaczonych do pracy podczas tzw. spacerów kosmicznych.

Sztuczne satelity można podzielić nie tylko ze względu na funkcjonalność ale także z uwagi na rozmiary, a dokładniej - masę startową:

- Minisatelity - od 100 do 500 kg,
- Mikrosatelity - od 10 do 100 kg,
- Nanosatelity - od 1 do 10 kg,
- Pikosatelity - od 0,1 do 1 kg.
- Femtosatelity - poniżej 100 g.

Jak widać, do kategorii satelitów małych można zaliczyć nawet relatywnie ciężkie urządzenia, umożliwiające świadczenie usług komercyjnych.

Z uwagi na wciąż duże koszty wiążące się z wystrzeleniem aparatury na orbitę okołoziemską, nowe firmy wkraczające w branżę space inwestują głównie w mniejsze

konstrukcje - nanosatelity.

Szczególnie obiecujący okazuje się przy tym standard znany jako CubeSat - jest to szczególna grupa nanosatelitów o masie rzędu około 1-2 kg oraz formie, będącej wielokrotnością “kostki jednostkowej"o wymiarach 100x100x113,5 mm [13], określanej mianem unit stąd też często spotykane oznaczenia typu 1U, 2U, 3U bądź 6U.

Najczęściej stosowane są konstrukcje użytkowe mające wymiary na poziomie 3U podczas gdy mniejsze satelity (głównie 1U, rzadziej 2U) są chętnie opracowywane przez zespoły studenckie z wyższych uczelni technicznych czy nawet amatorów elektroniki.

Ograniczenia masowe i wymiarowe popychają konstruktorów do stosowania niezwykle interesujących rozwiązań, umożliwiających zwiększenie funkcjonalności miniaturowego satelity już po jego wystrzeleniu w otwartą przestrzeń kosmiczną.

Otwarcie dostępu do przestrzeni kosmicznej - przede wszystkim niskiej orbity okołoziemskiej - dla małych i średnich firm oraz wyższych uczelni technicznych powoduje, że rośnie liczba przedsiębiorstw oraz projektów badawczych związanych z branżą space.

## 1.2 Przegląd aktualnych rozwiązań

### 1.2.1 PW-SAT2

PW-Sat2 to innowacyjny projekt studentów Politechniki Warszawskiej, którego głównym celem technologicznym było skonstruowanie i przetestowanie autorskiego systemu deorbitacji. Podczas misji zostały przetestowane także czujnik Słońca oraz system otwieranych paneli słonecznych.

Studenci PW wpadli na pomysł, by po zakończeniu swojej planowej misji satelita rozwinął specjalny żagiel, który stopniowo obniżając orbitę - spowoduje samoczynne spłonięcie satelity w atmosferze. Dzięki zastosowaniu odpowiedniej konstrukcji opartej na sprężynach płaskich oraz supercienkiej folii mylarowej, powierzchnia w pełni rozłożonego żagla wynosiła około $4m^2$. Tak duża powierzchnia poprzez zwiększenie oporu aerodynamicznego skutecznie spowolniła ruch satelity i spowodowała jego deorbitację.

### 1.2.2 ARDUSAT

W 2013 roku wystrzelono w przestrzeń kosmiczną dwa nanosatelity bazujące na platformie Arduino ArduSat-1 i ArduSat-X. Dwa bliźniacze satelity o rozmiarze 1U były wyposażone w komputery pokładowe o architekturze hierarchicznej (jeden procesor główny ATmega2561 zarządzający aż szesnastoma podrzędnymi mikrokontrolerami ATmega328P, przeznaczonymi do realizacji poszczególnych eksperymentów).

Na pokładzie niepozornych konstrukcji znalazły się liczne czujniki, w tym 3-osiowe magnetometry, akcelerometry i żyroskopy, pirometry MLX90614, półprzewodnikowe czujniki temperatury TMP102, a także sensory natężenia światła TSL2561, podwójne liczniki Geigera, spektrometry optyczne oraz 1,3-megapikselowe kamery. CubeSaty komunikowały się z Ziemią za pomocą półdupleksowych transceiverów UHF w paśmie 435...438 MHz.

### 1.2.3 FossaSat-1

FossaSat-1 to kolejne osiągnięcie w technologii pikosatelitarnej. Dzięki swoim rozmiarom wynoszącym zaledwie 5 x 5 x 5 cm i wadze zaledwie 250 g jest jednym z najmniejszych satelitów, jakie kiedykolwiek wystrzelono w przestrzeń kosmiczną.

FossaSat-1 zapewnia bezpłatną komunikację IoT typu open source na całym świecie przy użyciu niedrogich modułów radiowych LoRa, dzięki którym studenci i amatorzy radiotechniki mogą komunikować się z satelitą.

## 1.3 Cel pracy

Celem pracy było opracowanie oraz wykonanie od podstaw modelu mikrosatelity zdolnego do odbycia lotu na balonie stratosferycznym w celu dokonania podstawowych pomiarów oraz przetestowania sposobów komunikacji na odległość.

Głównym motywem była ciekawość, chęć poszerzenia wiedzy i umiejętności a także zainteresowanie technologiami kosmicznymi, informatyką oraz elektroniką. Zdecydowałem się na samodzielne rozwiązanie z racji wysokich kosztów rozwiązań komercyjnych.

Ciągły rozwój elektroniki użytkowej i możliwości technologiczne pozwalają na budowę tego typu satelit coraz mniejszym kosztem - dysponując budżetem 30 tys. euro jest możliwym zbudowanie i wystrzelenie mikrosatelity na orbitę LEO.

Przez coraz większą dostępność technologii space-grade powstały nawet specjalne serwisy oferujące sprzedaż części i modułów do satelit jednak ich ceny nadal są bardzo wysokie i często nieosiągalne bez sponsora lub innej formy finansowego wsparcia.

Zmotywowało to mnie do zaprojektowania i zbudowania modelu satelity formatu CubeSat będącego w stanie odbyć co najmniej lot balonem stratosferycznym.

## 1.4 Zakres pracy

W niniejszej pracy poza w pełni samodzielnym projektem i wykonaniem skorzystałem w pewnym stopniu z istniejących już rozwiązań - najczęściej sprawdzonych - przeprowadzając na nich ewentualne modyfikacje.

Zakres pracy obejmuje trzy główne etapy: software, hardware oraz dokumentację.

### 1.4.1 Dokumentacja

- Spis wymagań funkcjonalnych i niefunkcjonalnych
- Diagram działania systemu operacyjnego
- Schemat obwodów elektronicznych modelu
- Zdefiniowanie modelu danych do komunikacji z modelem

### 1.4.2 Strona programowa

- Napisanie głównego programu kontrolującego pracę modelu
- Napisanie programu do obsługi stacji naziemnej do zdalnej komunikacji
- Testy jednostkowe i funkcjonalne

### 1.4.3 Strona sprzętowa

- Stworzenie obwodów drukowanych PCB
- Montaż podzespołów
- Testy sprzętu

Przejście wszystkich etapów powinno dać w wyniku operacyjny model satelity zdolny do prowadzenia pomiarów podstawowych wielkości fizycznych oraz umożliwiającym komunikację w trybie half-duplex w celu kontroli modelu oraz przesyłania zgromadzonych danych.

Dodatkowo w ramach pracy zaprojektowałem i skonstruowałem też prostą stację naziemną pozwalająca na komunikację z modelem na odległość.

## 1.5 Struktura pracy

Praca składa się z XX rozdziałów. Rozdział pierwszy poza wprowadzeniem do tematyki mikrosatelitów i technologii kosmicznej, oraz opisem wybranych projektów o podobnej tematyce i funkcjonalności, zawiera także zakres działań przeprowadzonych w celu spełnienia stawianych założeń.

W rozdziale drugim zdefiniowałem wymagania zarówno funkcjonalne jak i niefunkcjonalne przedstawianego projektu, przedstawiłem użyte narzędzia oraz technologie, oraz zaprezentowałem schemat blokowy modelu.

Rozdział trzeci zawiera opis projektu od strony sprzętowej, czyli wybór elementów elektronicznych, wybrane schematy obwodów oraz opis wykonania.

Na rozdział czwarty składają się opisy i analizy poszczególnych elementów oprogramowania sterującego.

W rozdziale piątym znajduje się prezentacja przykładowych danych zgromadzonych podczas testu funkcjonalnego modelu oraz analiza wyników.

Ostatni rozdział to całokształt wykonanych prac, a także wyciągnięte przeze mnie wnioski podczas procesu tworzenia modelu.

# Projekt

## 2.1 Wprowadzenie

Całość projektu tworzą dwa ściśle połączone ze sobą moduły:

- sprzętowy - w jego skład wchodzą: procesor, który kontroluje działanie pozostałych podsystemów, a także zestaw czujników zbierających dane fizyczne, moduł komunikacji odpowiadający za przesyłanie danych do i z satelity, moduł GPS służący do określenia pozycji oraz system zasilania. Całość jest połączona za pomocą obwodów drukowanych i zamknięta w obudowie formatu 1U.
- programowy - jego główną częścią jest oprogramowanie pracujące w pamięci procesora, za pośrednictwem portów IO i szyn komunikacyjnych jest w stanie zarządzać częścią sprzętową projektu, realizować operacje odczytu i zapisu do pamięci a także odpowiadać na pakiety komunikacyjne.

W poniższym rozdziale omówiłem niektóre aspekty implementacji projektu.

## 2.2 Wymagania funkcjonalne

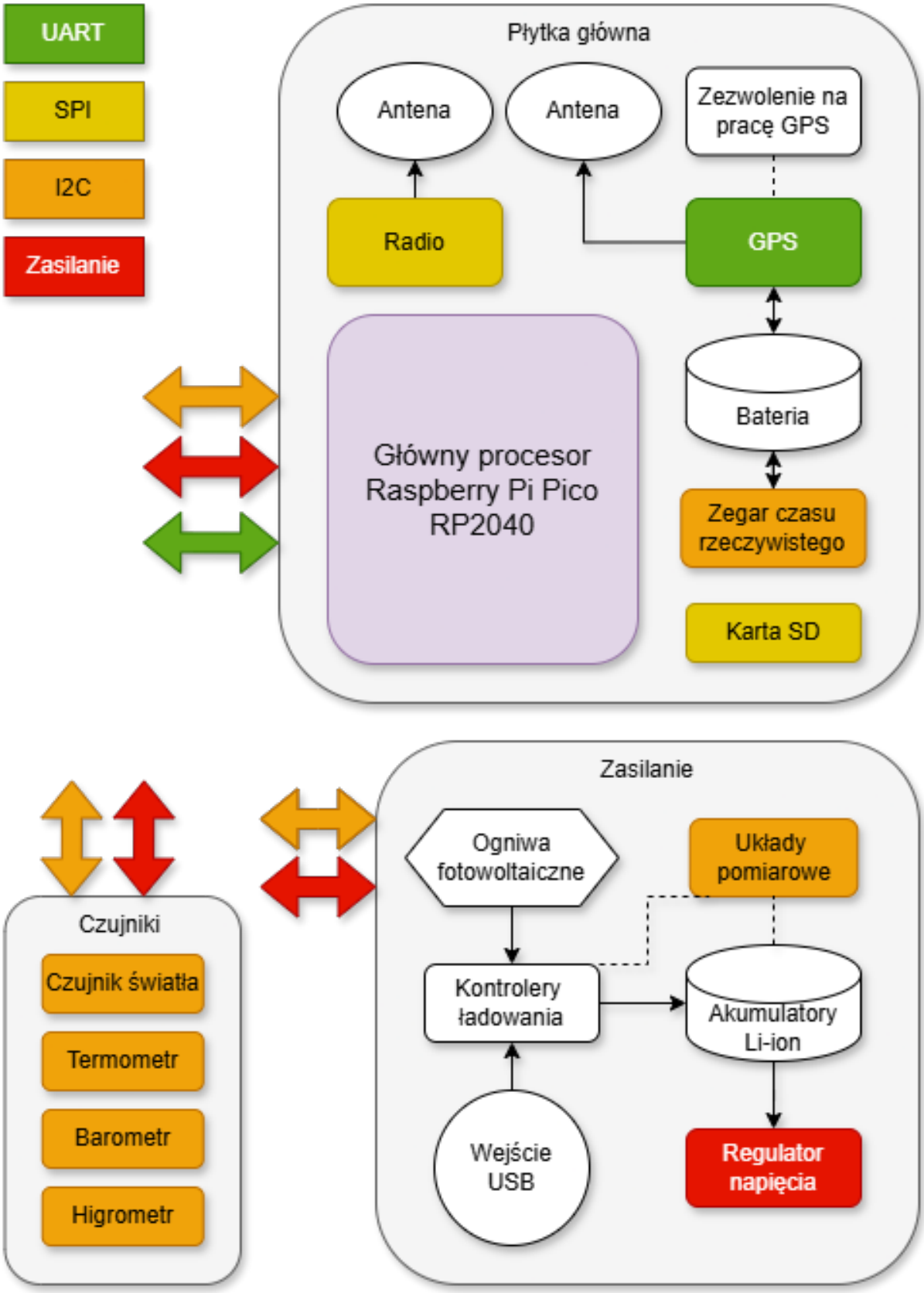
System musi spełniać następujące wymagania funkcjonalne:

- Wysyłanie danych z pomiarów do stacji naziemnych na żądanie - jednorazowo lub cyklicznie
- Możliwość zmiany konfiguracji poprzez zdalne polecenie
- Odpowiednie reagowanie i odpowiadanie na polecenia
- Rejestrowanie danych z czujników i podsystemów do pamięci nieulotnej
- Podstawowa weryfikacja i kontrola poprawności przesyłanych danych
- Wykrywanie i raportowanie anomalii w systemie oparte na systemie zdarzeń
- Wszystkie operacje muszą być wykonywane w określonym czasie, np. przesył danych nie może trwać dłużej niż 2 sekundy.

## 2.3 Schemat blokowy modelu

Rysunek poniżej przedstawia ogólny schemat budowy modelu od strony sprzętowej. Cały system jest podzielony na mniejsze, częściowo niezależne sekcje:

- Procesor - jego głównymi zadaniami jest kontrola układów podrzędnych, obłsuga pamięci oraz kodowanie i dekodowanie pakietów komunikacji modułu radiowego.
- Zasilanie - w skład sekcji wchodzą ogniwa fotowoltaiczne, ładowarki, kontroler zasilania oraz magazyn energii.
- Pamięć - na tą część składa się karta pamięci SD która rejestruje dane telemetryczne, pomiary z czujników oraz zdarzenia systemowe
- Komunikacja - podstawą tego podsystemu jest moduł radiowy działający w trybie half-duplex oraz antena - służą do wysyłania i odbierania danych telemetrycznych, kontrolnych oraz konfiguracyjnych
- Czujniki - ta sekcja składa się z zestawu czujników mających za zadanie dokonywać pomiarów różnych wielkości fizycznych w trakcie lotu




Rysunek 1: Schemat blokowy

## 2.4 Wykorzystane narzędzia

Do implementacji oprogramowania użyłem środowiska programistycznego Visual Studio Code firmy Microsoft które jest wszechstronnym IDE wspierającym proces tworzenia oprogramowania na mikrokontrolery oraz pozwalającym na sprawne debugowanie i monitorowanie działania aplikacji. Kompilacja odbywa się przy użyciu CMake. Za dostarczenie bibliotek do obsługi peryferiów mikrokontrolera odpowiada Pi Pico SDK. W trakcie powstawania projektu korzystałem także z platformy GitHub jako systemu kontroli wersji w celu łatwego kontrolowania zmian zachodzących w strukturze kodu.

Fusion 360 stworzony przez Autodesk były kluczowy w procesie projektowania modelu 3d satelity. Do projektowania układów elektronicznych i obwodów drukowanych użyłem programu KiCad który jest open-source, a dzięki dużej dostępności bibliotek oraz rozbudowanej funkcjonalności pozwolił sprawnie wykonać projekt. Zarządzanie projektem zrealizowałem przy użyciu Trello produkcji Atlassian oraz funkcji Github Issues, co umożliwiło łatwą organizację zadań, kontrolę nad postępami w projekcie oraz śledzenie bugów i wprowadzanie poprawek.

Wydrukowanie obudowy satelity oraz wyprodukowanie obwodów drukowanych zrealizowałem korzystając z oferty firmy JLCPCB, która oferuje usługi wytwarzania obwodów drukowanych w praktycznie dowolnej konfiguracji oraz drukowania 3D w wielu technologiach.

# Warstwa sprzętowa

Dobór elementów elektronicznych do prezentowanego projektu nie był łatwym zadaniem ponieważ wymagał głębszej analizy specyfikacji poszczególnych układów i elementów dyskretnych w celu spełnienienia większych wymagań niż typowa naziemna elektronika użytkowa. Za cel stawiając sobie stworzenie modelu zdolnego odbyć lot balonem stratosferycznym konieczne było dobranie elementów zdolnych do pracy w szerszym zakresie temperatur. Biorąc pod uwagę wysokość nad poziomem morza na jaką jest w stanie wznieść się tego typu balon odbiornik GPS również musi mieć większe możliwości między innymi zdolność pracy do wysokości 50000 m n.p.m.

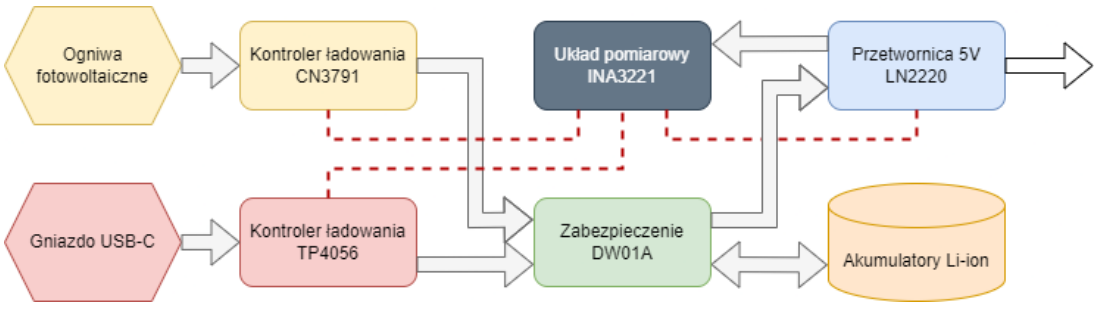
Większa wysokość oznacza również niższą temperaturę co z kolei wymaga bardziej dokładnego zaprojektowania układu zasilania oraz izolacji termicznej całej obudowy tak aby cenne ciepło zostawało w środku. Kluczowym podzespołem są akumulatory, których pojemność i sprawność drastycznie spada wraz ze spadkiem temperatury.

Warstwę sprzętową podzieliłem na 3 osobne płytki będące niezależnymi modułami.

## 3.1 Moduł zasilania

W skład tego modułu wchodzą akumulatory wraz z zabezpieczeniami, układy ładowania, przetwornica napięcia oraz układ pomiarowy.

Interfejs tej płytki stanowi gniazdo JST 6 pin w którym umieściłem sygnały szyny I2C do komunikacji z układem pomiarowym, napięcie wyjściowe przetwornicy 5V oraz dwa piny potencjału masowego. Poniżej umieściłem schemat przedstawiący przepływ energii oraz zależności między poszczególnymi podzespołami.




Rysunek 2: Diagram modułu zasilania

Magazyn energii stanowią dwa akumulatory litowo-jonowe w powszechnym formacie 18650 o napięciu nominalnym 3.7V i pojemności 2000mAh. Łacząc je szeregowo uzyskałem łączną pojemność na poziomie 4000mAh.

Podczas pracy procesora z akumulatorów pobierany jest prąd o wartości około 40mA co pozwala na teoretyczny czas pracy na poziomie 100 godzin. W praktyce czas ten jest krótszy ze względu na inne układy pobierające prąd oraz straty w przetwornicy napięcia. Moduł GPS skonfigurowany w trybie High-Performance mode pobiera prąd o wartości około 80mA co w sumie daje 120mA - co skraca czas pracy 33 godzin. W praktyce przelot balonu stratosferycznego trwa około 3 godzin co pozwala na zapewnienie zasilania przez cały lot.

Korzystając z tego faktu zaprojektowałem system zasilania tak aby mógł być ładowany na ziemi przez gniazdo USB typu C. Projekt przewiduje montaż paneli fotowoltaicznych na obudowie satelity w celu zbadania możliwości ładowania akumulatorów w trakcie lotu przez pomiar prądów i napięć oraz późniejszą analizę tych danych.

### 3.1.1 Układ pomiaru napięć i prądów

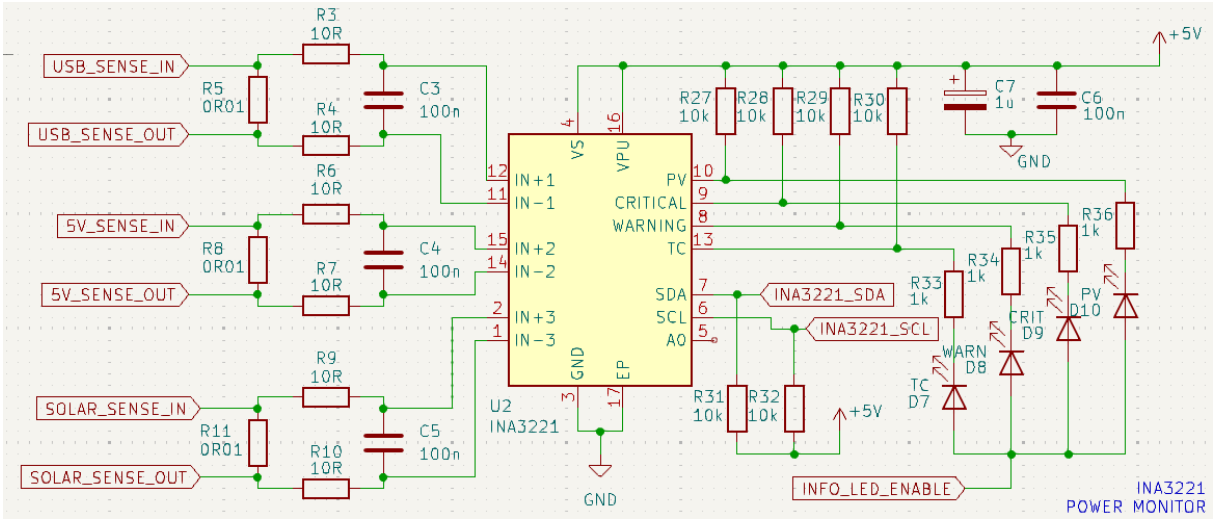
W trakcie pracy konieczny jest pomiar napięcia baterii oraz napięcia przetwornicy 5V, a także prądów ładowania i rozładowania. Do tego celu użyłem układu INA3221, który pozwala na pomiar napięć i prądów na trzech niezależnych kanałach. [6] Układ ten jako jedyny jest zasilany napięciem 5V a komunikacja z układem sterującym odbywa się po szynie I2C - konieczny jest konwerter poziomów logicznych z 5V na 3.3V ponieważ na takim poziomie napięć operuje procesor.

INA3221 umożliwiaj pomiar prądu przez pmiar spadku napięcia na rezystorach bocznikowych. Układ charakteryzuje się wysoką dokładnością pomiaru ($\pm 0.5\%$) oraz niskim poborem prądu (około 1mA w trybie pracy ciągłej). Zastosowałem następujące konfiguracje kanałów pomiarowych:

| Kanał | Opis |
|---|---|
| 1 | Napięcie baterii i ładowanie USB |
| 2 | Napięcie przetwornicy 5V |
| 3 | Prąd ładowania z paneli fotowoltaicznych |

Tabela 3.1: Konfiguracja kanałów pomiarowych INA3221

Dzięki rejestrowi konfiguracyjnemu możliwe jest ustawienie liczby uśrednień dla pojedynczego pomiaru (od 1 do 1024), co pozwala na zwiększenie dokładności kosztem częstotliwości akwizycji danych. W moim projekcie zastosowałem uśrednianie 16 próbek w ciągłym trybie pracy, co zapewnia dobrą równowagę między dokładnością a szybkością pomiaru.




Rysunek 3: Schemat układu pomiarowego

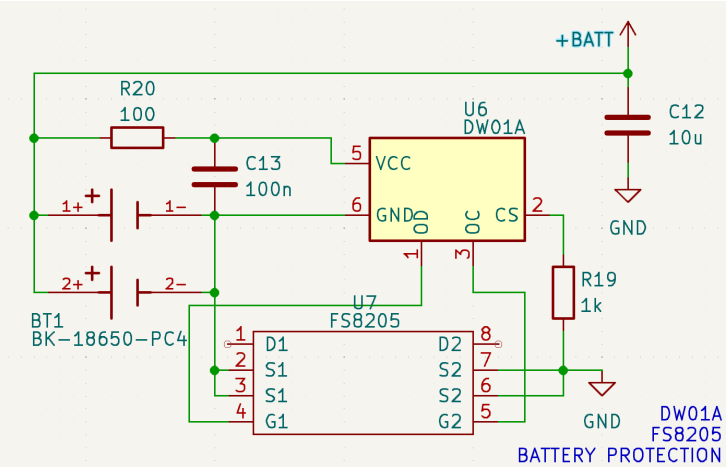
### 3.1.2 Zabezpieczenie akumulatorów

Do zabezpieczenia akumulatorów użyłem układu DW01A będącego jednocześnie zabezpieczeniem przed nadmiernym naładowaniem oraz rozładowaniem. Minimalne napięcie akumulatorów jest ograniczone do 2.4V a maksymalne do 4.3V. Zachowując napięcie akumulatorów między tymi dwiema wartościami skutecznie chroni je to przed degradacją i uszkodzeniem.

Układ DW01A współpracuje z podwójnym tranzystorem MOSFET FS8205A, tworząc pełne zabezpieczenie akumulatorów. Tranzystor FS8205A zawiera dwa kanały typu N połączone szeregowo źródłami, co tworzy przełącznik dwukierunkowy umożliwiający odcięcie przepływu prądu zarówno podczas ładowania jak i rozładowywania. Taka konfiguracja eliminuje potrzebę stosowania dodatkowych diod zabezpieczających.

W normalnym trybie pracy układ DW01A pobiera jedynie około 3 $\mu A$, co minimalizuje jego wpływ na samorozładowanie akumulatorów [3] Dzięki zastosowaniu obudowy

SOT-23-6 zajmuje również niewiele miejsca na płytce PCB, co jest istotne przy kompaktowej konstrukcji urządzenia.

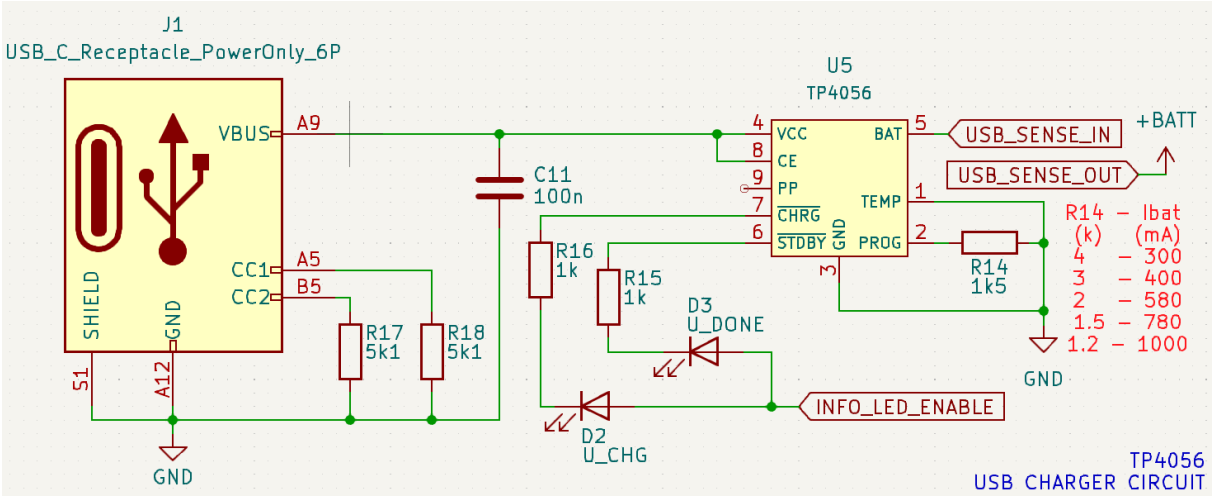



Rysunek 4: Schemat zabezpieczenia akumulatorów

### 3.1.3 Ładowarka 1 (USB)

W celu ładowania akumulatorów przez gniazdo USB typu C wykorzystałem powszechnie stosowany układ TP4056, jego dostępność jest bardzo wysoka i bardzo łatwo go zintegrować do własnego rozwiązania. Maksymalne napięcie zasilania TP4056 to 8V jednak piny sterujące CC1 i CC2 gniazda USB skonfigurowałem tak aby napięcie było równe 5V. Prąd ładowania jest konfigurowalny w zakresie 130-1000mA. Aby ograniczyć wydzielanie ciepła w trakcie ładowania ograniczyłem je do około 780mA.

Układ TP4056 implementuje algorytm ładowania CC/CV (stały prąd/stałe napięcie) odpowiedni dla akumulatorów litowo-jonowych. W pierwszej fazie ładowania dostarcza stały prąd ustawiony za pomocą rezystora $R_{PROG}$, po czym przechodzi do fazy stałego napięcia 4.2V, gdy akumulator osiągnie odpowiedni poziom naładowania. Efektywność energetyczna układu wynosi około 80%, co jest typową wartością dla liniowych ładowarek tego typu.




Rysunek 5: Schemat układu ładowania USB

### 3.1.4 Ładowarka 2 (panele fotowoltaiczne)

Do ładowania akumulatorów z paneli fotowoltaicznych zastosowałem układ CN3791. Jest to kontroler ładowania MPPT (Maximum Power Point Tracking), który automatycznie dostosowuje punkt pracy paneli fotowoltaicznych tak, aby uzyskać maksymalną możliwą moc. Układ może pracować z napięciem wejściowym od 4.5V do 28V, co pozwala na wykorzystanie szerokiej gamy paneli słonecznych.

CN3791 również realizuje algorytm ładowania CC/CV, jednak w przeciwieństwie do TP4056 jest to przetwornica impulsowa typu buck, co zapewnia większą sprawność energetyczną przy konwersji napięcia (do 96%). Napięcie pracy MPPT ustawiłem na poziom 4.9V, co jest optymalne dla zastosowanych paneli fotowoltaicznych.

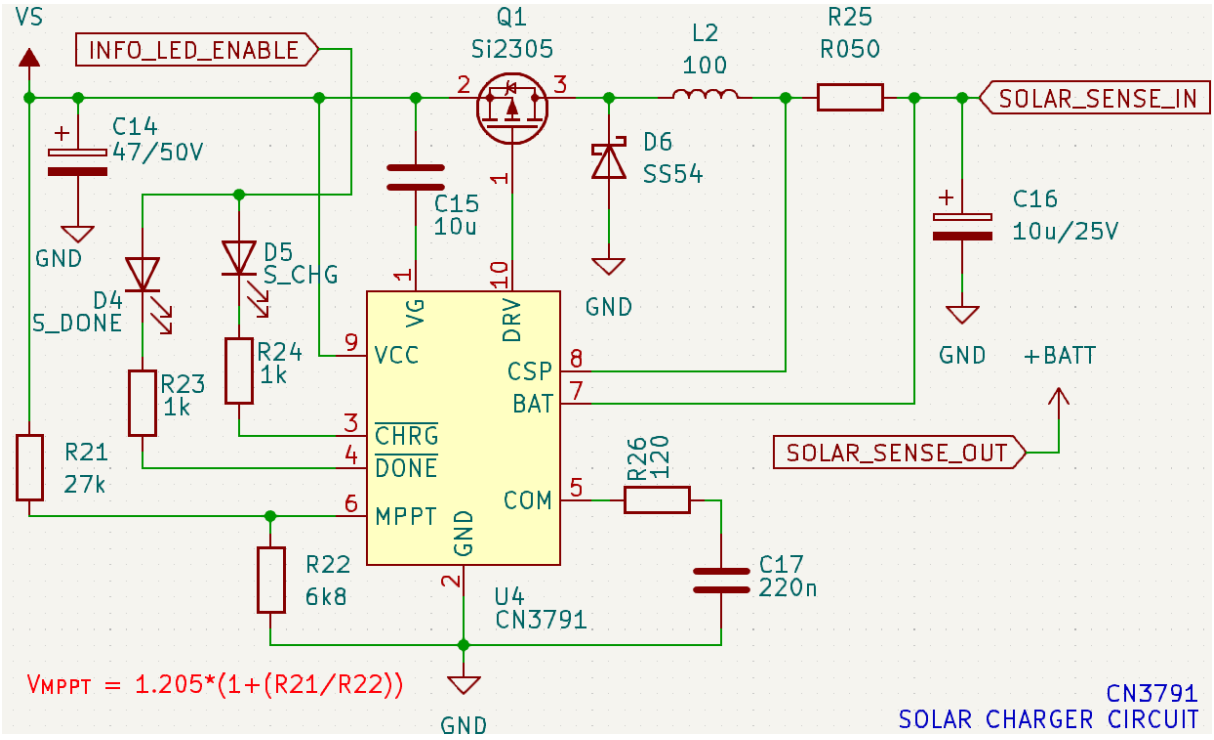
Do sprawdzenia możliwości ładowania akumulatorów użyłem ogniw o następujących parametrach:

| **Parametr** | **Wartość** |
|---|---|
| Napięcie znamionowe | 5,5 V |
| Prąd znamionowy | 110 mA |
| Moc szczytowa | 0,55 W |

Tabela 3.2: Parametry zastosowanego panelu fotowoltaicznego

Obudowę satelity zaprojektowałem tak aby umożliwić montaż 4 paneli fotowoltaicz-

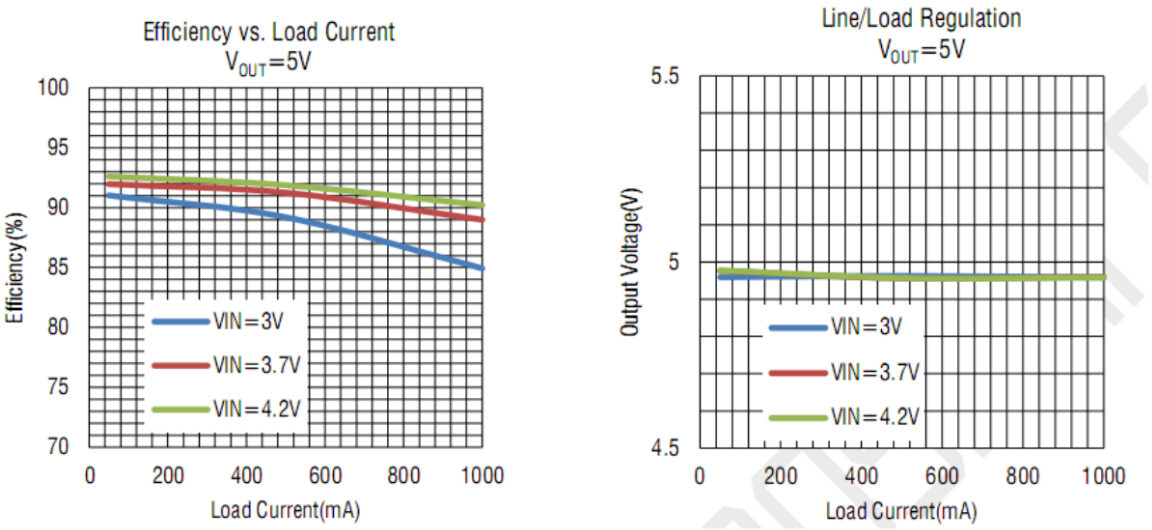
nych, po jednym na każdej ścianie. Podstawa obudowy ma plan kwadratu co oznacza że maksymalną wydajność mogą osiągnąć maksymalnie 2 panele jednocześnie. Ogniwa zostały połączone szeregowo w celu zwiększenia napięcia wyjściowego, a następnie podłączone do układu CN3791.




Rysunek 6: Schemat układu ładowania paneli fotowoltaicznych

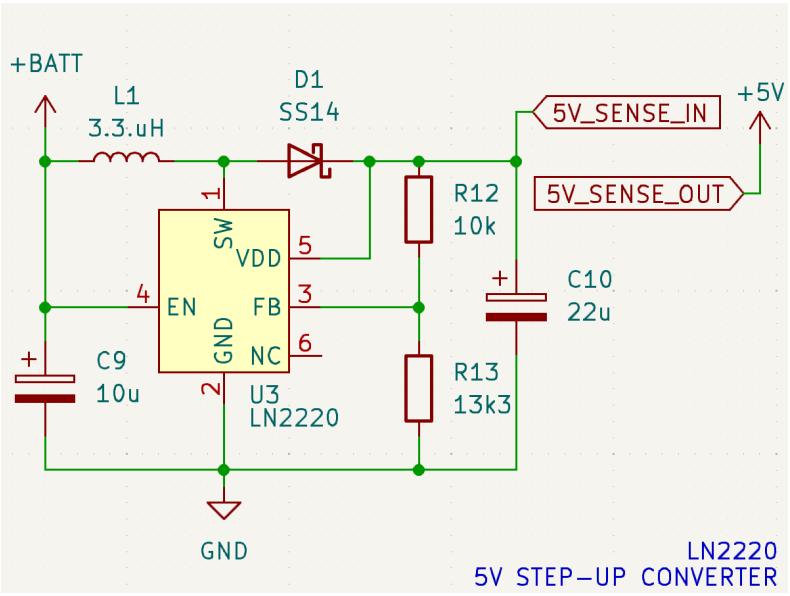
### 3.1.5 Przetwornica 5V

Do konwersji napięcia akumulatorów (3.7V nominalnie) na stabilne napięcie 5V wymagane przez układ pomiarowy INA3221 oraz inne komponenty systemu zastosowałem przetwornicę impulsową opartą na układzie LN2220. Jest to synchroniczna przetwornica buck charakteryzująca się wysoką sprawnością (do 92%) przy obciążeniach typowych dla projektowanego systemu [10]. Na poniższych diagramach producenta można zauważyć że przy obciążeniu nieprzekraczającym 150mA sprawność pozostaje na poziomie 90% z niewielkimi spadkami przy niższych napięciach wejściowych.




Rysunek 7: Charakterystyka przetwornicy LN2220

Układ LN2220 pracuje z częstotliwością przełączania 1.2MHz, co pozwala na zastosowanie mniejszych elementów pasywnych, a tym samym zmniejszenie wymiarów płytki PCB. Maksymalny prąd wyjściowy przetwornicy wynosi 2A, co z dużym zapasem pokrywa wymagania systemu, którego typowy pobór prądu wynosi około 120mA. Stabilność napięcia wyjściowego w zakresie 5V $\pm 2\%$ jest zapewniona przez sprzężenie zwrotne, a dodatkowo układ wyposażony jest w zabezpieczenia przed zwarciami, przegrzaniem oraz zbyt niskim napięciem wejściowym.




Rysunek 8: Schemat przetwornicy

## 3.2 Moduł sterujący

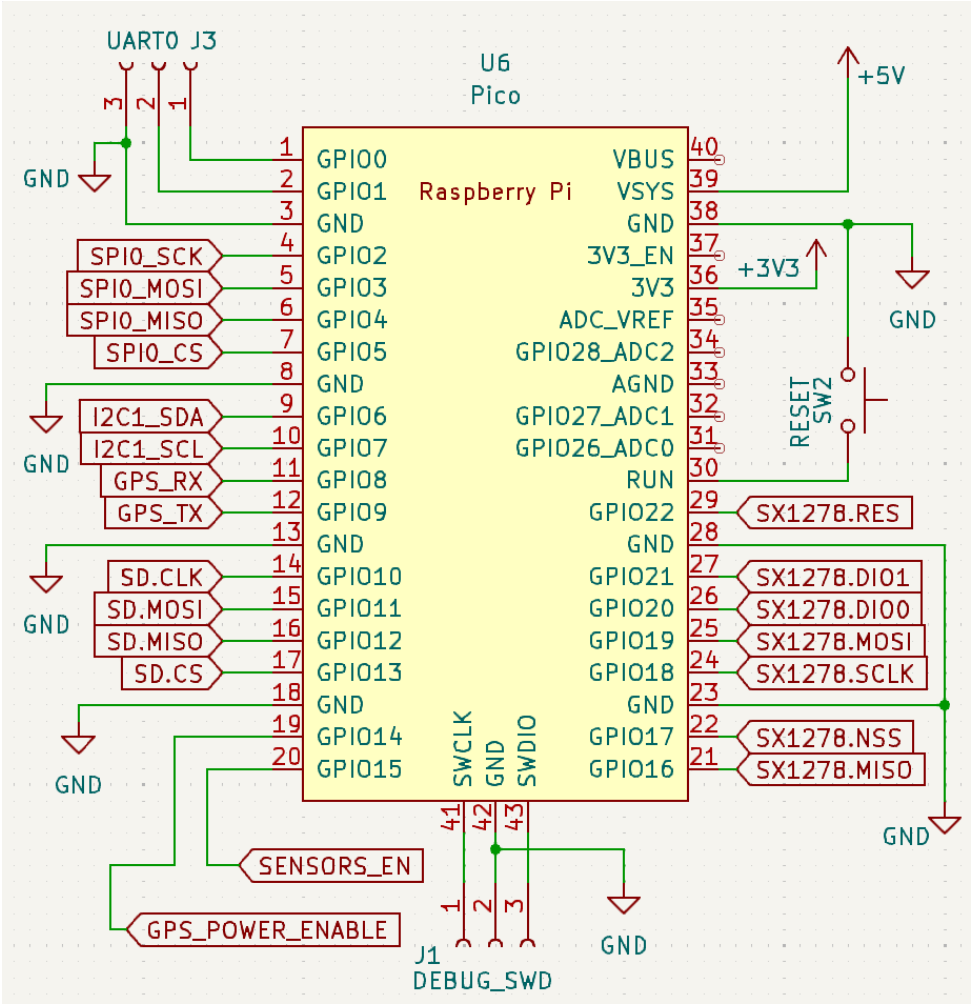
Sekcję kontroli i sterowania zbudowałem w oparciu o płytkę prototypową Raspberry Pi Pico, wspomaganą odbiornikiem GPS, modułem radiowym LoRa pracującym w trybie half-duplex, zegarem RTC oraz kartą SD do przechowywania danych. Praca całego programu jest podzielona na dwa wątki, po jednym na każdy rdzeń procesora.

Rdzeń 0 odpowiada tylko za obsługę komunikacji - odbieranie, przetwarzanie i wysyłanie ramek komunikacji 4.2. Rdzeń 1 odpowiada za zbieranie danych z GPS oraz dane telemetryczne - pomiary z czujników, napięć i prądów oraz zapisywanie ich na kartę SD.

Wyprowadziłem osobne złącza zawierające magistrale I2C, SPI oraz dodatkowe zasilanie - służą one do podłączenia płytki z czujnikami. Uruchomienie systemu odbywa się poprzez zasilenie płytki Pi Pico z wewnętrznego zasilania lub z zewnętrznego źródła 5V. Sekwencja startowa została opisana w 4.1.1

### 3.2.1 Procesor

Głównym procesorem jest dwurdzeniowy mikrokontroler RP2040 produkowany przez Raspberry Pi Foundation. Jest on zbudowany w oparciu o architekturę ARM Cortex M0+ z taktowaniem 133MHz. Jest wyposażony w 264kB pamięci operacyjnej, podwójne interfejsy UART, SPI oraz I2C. Zaletami układu są jego możliwości, dostępność, cena a także popularność i wsparcie które oferuje sam producent oraz społeczność entuzjastów Raspberry. Przekłada się to na dużą dostępność materiałów, dokumentacji oraz dobrze przygotowane SDK, które poza tworzeniem kodu pozwala także na jego debugowanie w czasie rzeczywistym. Program aktualnie wykonywany przez procesor znajduje się w pamięci QSPI FLASH, w której można również umieścić inne dane jak konfiguracje, pomiary lub logi. Przetwornica dostarcza napięcia 3.3V wymaganego do zasilania procesora, pamięci oraz zewnętrznych podzespołów.




Rysunek 9: Schemat połączeń procesora

### 3.2.2 Moduł komunikacji radiowej

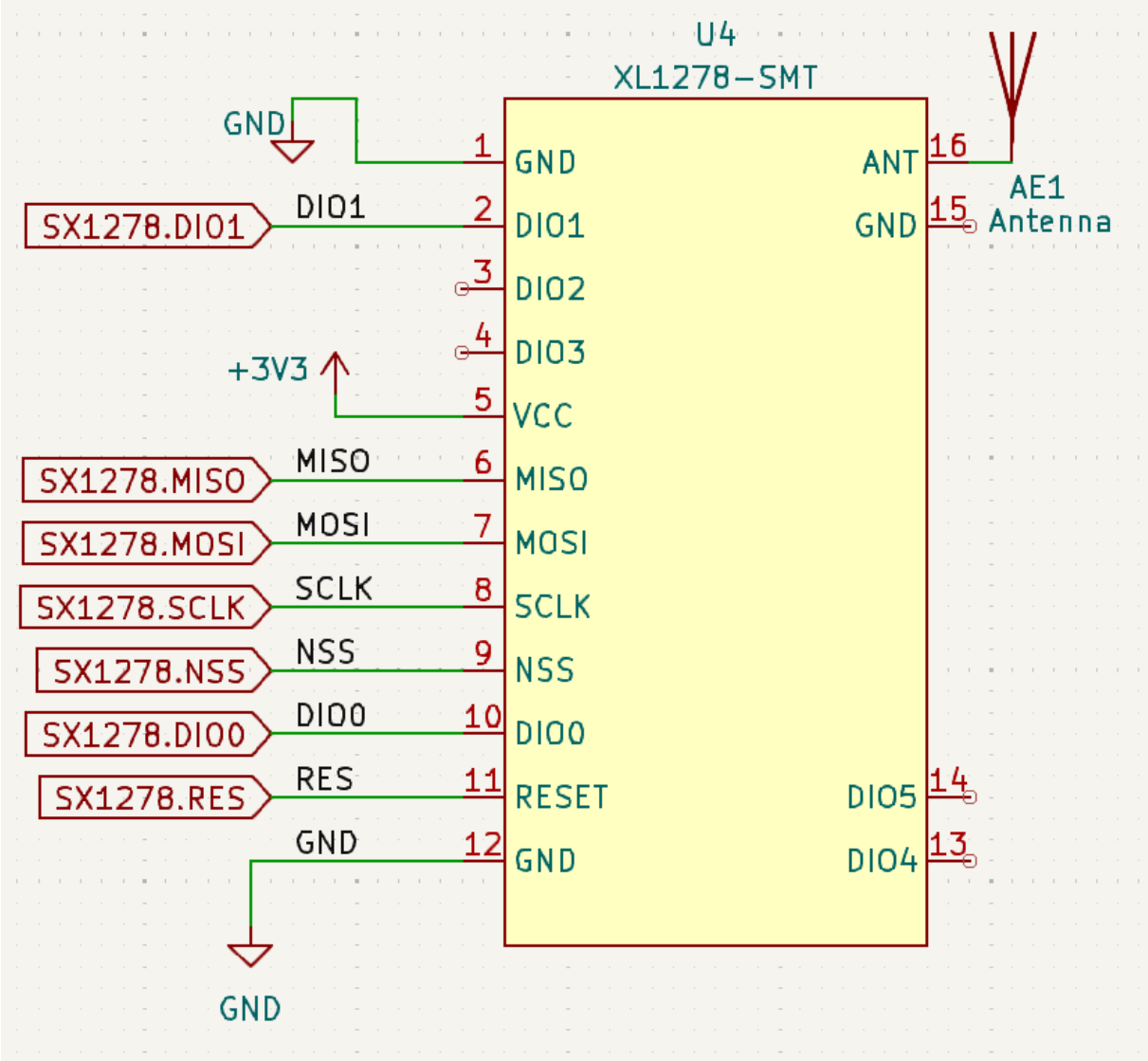
Odpowiada za komunikację modelu z stacją naziemną, może pracować z częstotliwością z zakresu 410-525MHz z maksymalną prędkością 300 kbps [8].

Moc wyjściowa to +18dBi co w otwartej przestrzeni pozwala na uzyskanie zasięgu do 700m jednak są dowody na udane przesyłanie danych na odległość ponad 700km. [11] Moduł LoRa są bardzo popularne w przemyśle i komunikacji urządzeń przenośnych, często są też wykorzystywane jako główny moduł komunikacyjny w satelitach typu CubeSat [5].

Zużycie prądu w stosunku do zasięgu jest bardzo niskie dlatego jest to idealny moduł do zastosowań gdzie zasilanie jest ograniczone np. w modelach latających. Dla pracy na częstotliwości 433MHz pobór prądu prezentuje się następująco:

| Stan | Pobór prądu [mA] |
|---|---|
| Odbiór | 12.1 |
| Nadawanie | 93 |
| Oczekiwanie | 1.6 |

Tabela 3.3: Pobór prądu modułu SX1278




Rysunek 10: Schemat połączeń modułu radiowego

### 3.2.3 Odbiornik GPS

Moduł GPS NEO-6M jest odbiornikiem GPS pracującym w paśmie L1 (1575.42 MHz), dane nawigacyjne są aktualizowane i przesyłane co sekundę przez port szere-

gowy UART [12]. Format danych to NMEA-0183, który jest standardem w nawigacji morskiej i lotniczej.

Służy do śledzenia pozycji modelu z precyzją 2.5m co po zakończeniu lotu pozwala na odnalezienie modelu w terenie. Z tego powodu zasilanie modułu GPS jest włączone na cały czas i w trybie lotu nie jest możliwe jego zdalne wyłączenie.

Zmiana stanu zasilania modułu jest realizowana przez miniaturową przetwornicę DC-DC, która dostarcza napięcie 3.3V z zasilania 5V. Posiada ona wejście EN (Enable) które pozwala na włączenie/wyłączenie w przetwornicy i w efekcie włączenie/wyłączenie modułu GPS. Wejście EN jest podłączone do pinu GPIO procesora, co pozwala na zmianę stanu zasilania modułu GPS z poziomu oprogramowania.

Moduł posiada wbudowaną antenę ceramiczną, która jest wystarczająca do uzyskania sygnału w otwartej przestrzeni.

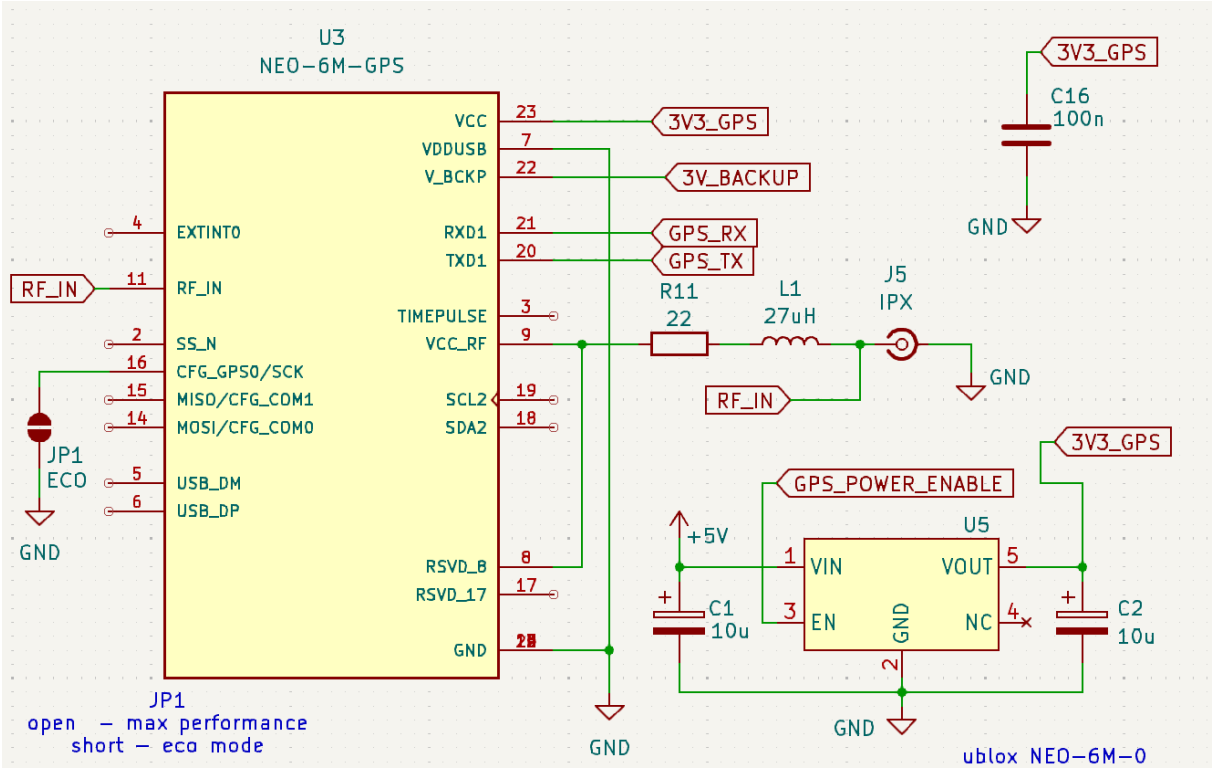
Zimny start [1] trwa około 30 sekund, następnie almanach i efemeryda zostają zapisane w pamięci modułu - przy kolejnych uruchomieniach modułu (hot start[2]) obliczenie pozycji zajmuje znacznie mniej czasu gdy dane są w miarę aktualne.

Dodatkowa bateria CR2032 zapobiega utracie danych w przypadku zaniku zasilania. Proces uzyskiwania lokalizacji przez moduł GPS przebiega następująco:

1. Inicjalizacja modułu.
2. Nasłuchiwanie sygnałów z widocznych satelitów (minimum 3 dla pozycji 2D, 4 dla pozycji 3D).
3. Dekodowanie danych nawigacyjnych zawierających efemerydy i almanachy.
4. Obliczenie pseudoodległości do każdego satelity na podstawie opóźnienia sygnału.
5. Triangulacja pozycji na podstawie danych z wielu satelitów.

[1] Zimny start oznacza uruchomienie modułu po całkowitym wyczyszczeniu danych nawigacyjnych - np. przy pierwszym uruchomieniu lub po rozładowaniu baterii podtrzymującej. W tym trybie moduł musi pozyskać pełne dane efemerydy od satelitów, co trwa zwykle ponad 30 sekund.

[2] Hot start to tryb uruchomienia, w którym moduł GPS posiada aktualne dane efemerydy, ważną pozycję, czas oraz almanach. W takim przypadku czas do uzyskania pierwszej poprawnej pozycji (TTFF) wynosi zaledwie 1-3 sekundy.




Rysunek 11: Schemat połączeń modułu GPS

### 3.2.4 Zegar czasu rzeczywistego

DS3231 to precyzyjny zegar czasu rzeczywistego z wbudowanym termometrem, komunikujący się przez interfejs I2C.

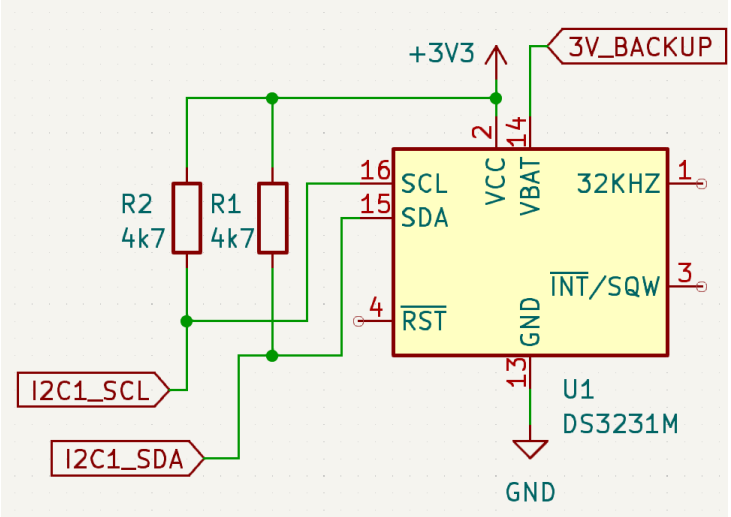
Ten moduł przechowuje czas w postaci godzin, minut i sekund oraz datę w postaci roku, miesiąca i dnia. Cechuje go wysoka dokładność - $\pm 3.5$ ppm w zakresie temperatur od $-40°$C do $+85°$C, co odpowiada odchyleniu maksymalnie około 9 sekund na miesiąc. Jest to możliwe dzięki wbudowanemu kompensowanemu temperaturowo oscylatorowi kwarcowemu (TCXO).

Zegar obsługuje format 12/24-godzinny oraz automatycznie uwzględnia różnice w długości miesięcy i lata przestępne, dzięki czemu nie wymaga ręcznego korygowania daty.

Do podtrzymania chodu zegara przy zaniku zasilania służy bateria CR2032 współdzielona z odbiornikiem GPS. Dzięki temu system zachowuje aktualną datę i czas nawet po wyłączeniu zasilania, co jest kluczowe dla prawidłowego znakowania czaso-

wego wszystkich zbieranych danych telemetrycznych.

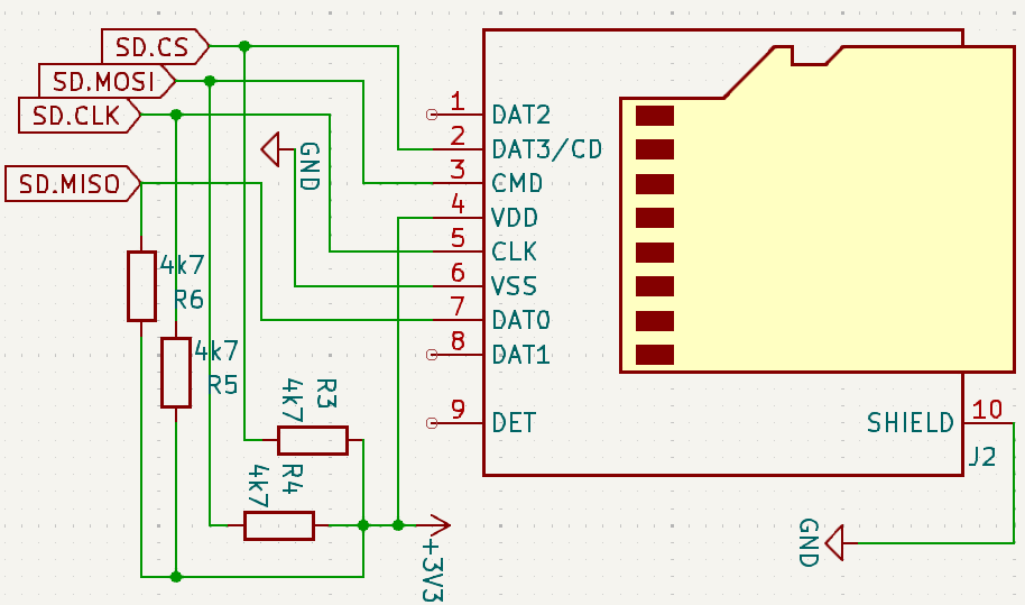
Moduł DS3231 jest konfigurowany przez mikrokontroler przy starcie systemu, a w razie potrzeby jego ustawienia mogą być zmieniane zdalnie poprzez odpowiednie komendy wysyłane przez łącze radiowe.




Rysunek 12: Schemat połączeń zegara czasu rzeczywistego

### 3.2.5 Pamięć

Jako magazyn na dane telemetryczne, pomiary z czujników oraz logi zdarzeń systemowych użyłem karty pamięci SD o pojemności 2GB. Zapisując dane na karcie, po zakończeniu lotu można w łatwy sposób odczytać wszystkie zebrane informacje. Karta korzysta ekskluzywnie z interfejsu spi1 co pozwala na szybki i niezakłócony dostęp do danych. Na karcie są rejestrowane dwa rodzaje danych: wszystkie pomiary z czujników środowiskowych, pomiary napięć i prądów, aktualna lokalizacja oraz data i godzina, a także zdarzenia np. obniżenie napięcia poniżej progu, zmiana zegaru itp. wraz z dokładną datą i godziną wystąpienia. Te dane można również odczytać zdalnie używając odpowiednich poleceń.




Rysunek 13: Schemat połączeń karty pamięci SD

## 3.3 Moduł czujników

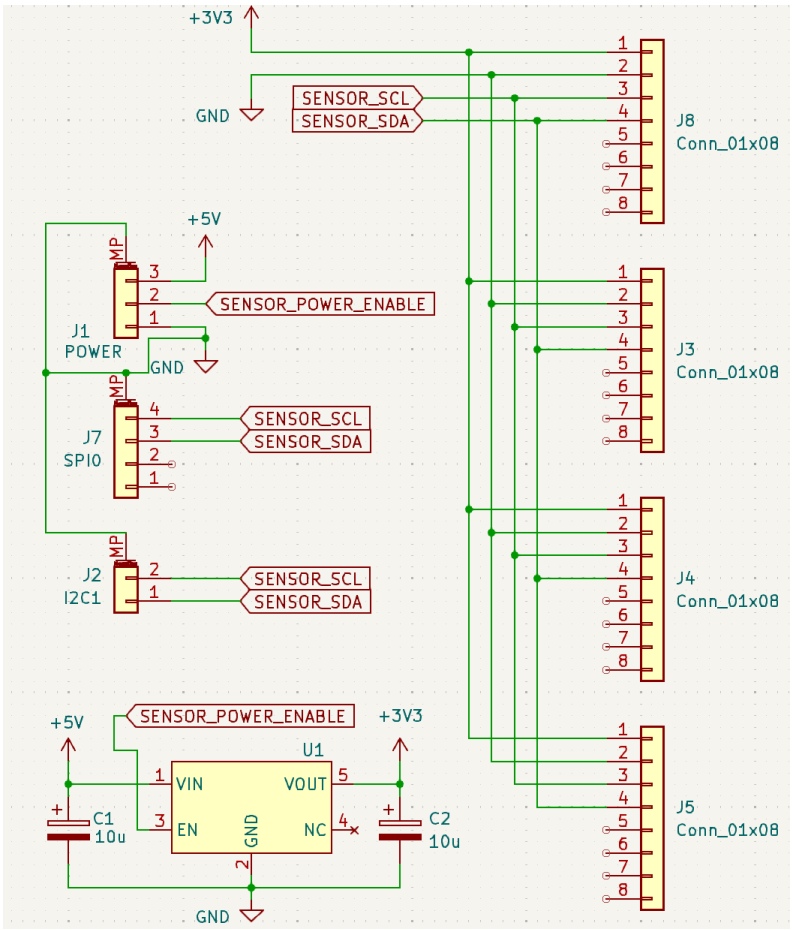
Poza wyżej wymienionymi model daje możliwość podłączenia szeregu różnych czujników środowiskowych, zarówno po szynie I2C jak i SPI. Płytka jest wyposażona w złącze zasilania 5V, złącze I2C oraz SPI. W bieżącej konfiguracji zdecydowałem się na 4 standardowe listy kołkowe o rastrze 2.54mm z 8 pinami każda, połączone z gniazdem I2C. Pozwala to na podłączenie 4 różnych czujników jednocześnie, co jest wystarczające do celów testowych. Oba moduły czujników mają identyczną kolejność wyprowadzeń co znacznie ułatwiło projektowanie oraz zadecydowało o wyborze takiej a nie innej konfiguracji. Schemat połączeń modułu czujników przedstawia rysunek 14.

### 3.3.1 Barometr, termometr, higrometr

W projekcie wykorzystałem czujnik BME280 firmy Bosch, który integruje w jednym układzie trzy podstawowe czujniki środowiskowe. Jest to układ o niewielkich wymiarach (2.5 x 2.5 x 0.93 mm), komunikujący się przez interfejs I2C lub SPI. W moim projekcie wybrałem komunikację I2C ze względu na mniejszą liczbę wymaganych linii.

BME280 oferuje następujące możliwości pomiarowe:

- Pomiar ciśnienia atmosferycznego w zakresie 300-1100 hPa z dokładnością $\pm 1$




Rysunek 14: Schemat połączeń modułu czujników

hPa

- Pomiar temperatury w zakresie -40℃ do +85℃ z dokładnością $\pm 1^{\circ}C$
- Pomiar wilgotności względnej w zakresie 0-100% z dokładnością $\pm 3\%$

Układ charakteryzuje się niskim poborem prądu (3.6 $\mu A$ przy pomiarze raz na sekundę) oraz szybkim czasem reakcji. Dzięki pomiarowi ciśnienia atmosferycznego możliwe jest określenie przybliżonej wysokości modelu nad poziomem morza, co stanowi uzupełnienie danych z GPS. Różnica ciśnienia o 1 hPa odpowiada w przybliżeniu zmianie wysokości o około 8,3 m na niskich wysokościach.

Dane z czujnika są odczytywane w regularnych interwałach czasowych i zapisywane na karcie SD oraz przekazywane w ramkach telemetrycznych do stacji naziemnej, dostarczając informacji o warunkach atmosferycznych otoczenia modelu podczas lotu.

### 3.3.2 Czujnik światła

Do pomiaru natężenia światła wykorzystałem czujnik cyfrowy BH1750 komunikujący się przez magistralę I2C. Jest to kompletny 16-bitowy przetwornik światła na wartość cyfrową z wbudowaną funkcją kompensacji czułości widmowej zgodnej z ludzkim okiem.

Czujnik BH1750 oferuje następujące możliwości:

- Szeroki zakres pomiarowy od 1 do 65535 luksów
- Pomiar z różną rozdzielczością (0.5 lx, 1 lx, 4 lx) zależnie od wybranego trybu
- Niewielki pobór prądu: 120 $\mu A$ podczas pomiaru, <1 $\mu A$ w trybie uśpienia
- Wbudowany filtr odpowiadający krzywej czułości spektralnej oka ludzkiego

Czujnik ten pozwala na monitorowanie warunków oświetleniowych podczas lotu, co może być użyteczne do rejestracji warunków środowiskowych. Ponadto, nagłe zmiany natężenia światła mogą sygnalizować przejście modelu z cienia w światło słoneczne lub odwrotnie, co może mieć znaczenie dla interpretacji danych z innych czujników, szczególnie temperatury.

Dane z czujnika światła są odczytywane w regularnych odstępach czasu i zapisywane wraz z pozostałymi danymi telemetrycznymi, co pozwala na ich późniejszą analizę w kontekście trasy lotu modelu.

## 3.4 Obudowa

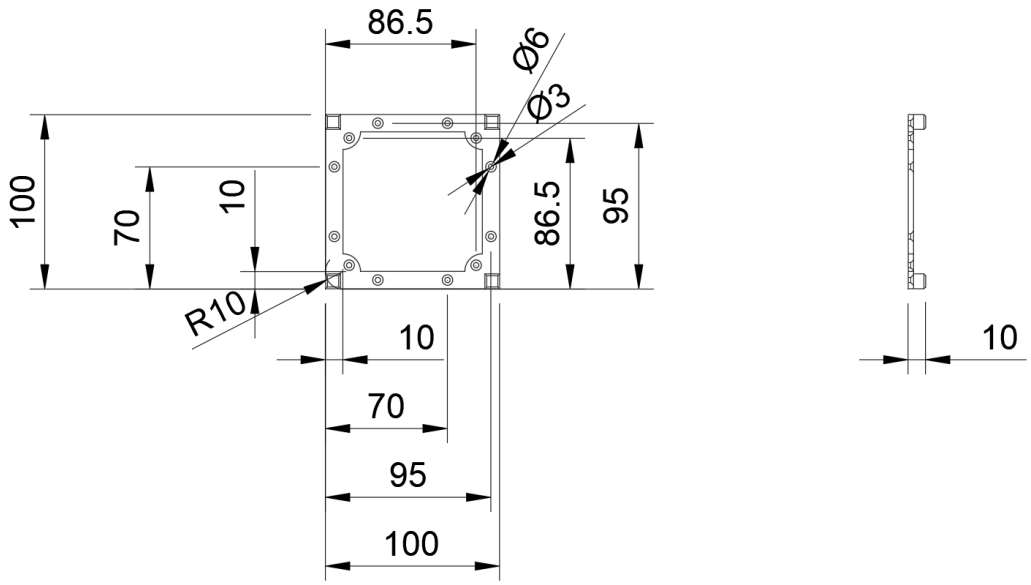
Obudowę zaprojektowałem w programie Autodesk Fusion 360, który pozwala na tworzenie modeli 3D oraz generowanie dokumentacji technicznej. Model obudowy składa się z 6 części: 4 elementów narożnych oraz 2 elementów podstawy. Wszystkie elementy zostały zaprojektowane tak, aby można je było wydrukować na drukarce 3D FDM. Zgodnie z wcześniejszymi założeniami zgodności ze standardem CubeSat, obudowa ma wymiary 100 x 100 x 113,5mm [13].

Podstawa ma grubość 10mm co jest kompromisem między wytrzymałością a ilością miejsca wewnątrz obudowy. Na każdym boku znajdują się otwory montażowe o średnicy 3mm, które umożliwiają przykręcenie elementów narożnych. Dodatkowo w pobliżu narożników znajdują się otwory o średnicy 3mm do montażu obwodów drukowanych. Pusty obszar w środku zapewnia łatwy dostęp do akumulatorów i gniazd ładowania od spodu, natomiast jako górna część obudowy pozwala na łatwy dostęp do modułów czujników i gniazd komunikacyjnych.

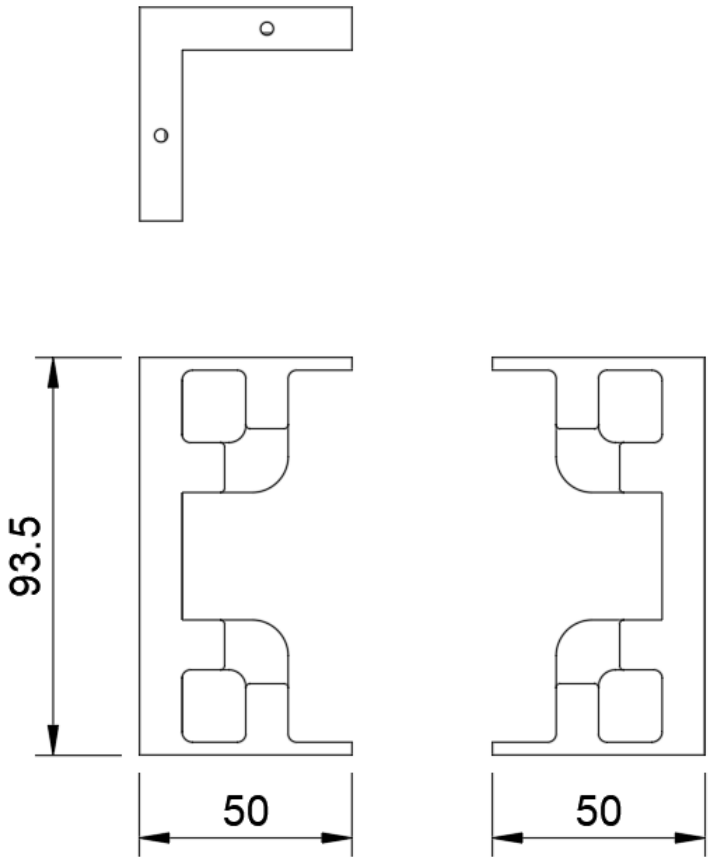
Każdy z modułów opisanych wcześniej (3.1, 3.2, 3.3) posiada na swojej płytce otwory montażowe, które umożliwiają przykręcenie ich do obudowy za pomocą poliamidowych elementów dystansowych o długości 22mm. Dobrałem je tak, aby suma ich długości oraz grubości płytki wynosiła 93.5mm dzięki czemu zniwelowałem ryzyko naprężeń w obudowie.

Obudowę można łatwo rozdzielić na dwie cześci co zapewniło łatwy dostęp do wnętrza w razie potrzeby co widać na renderach 17 oraz 19.

Dodatkowo po zmontowaniu między elementami narożnymi przewidziałem mocowania na ogniwa fotowoltaiczne o wymiarach 70x70x3mm. Render w pełni złożonego modelu bez ogniw przedstawiłem na rysunku 20. W pełni kompletny model znajduje się na rysunku 21.




Rysunek 15: Wymiary podstawy



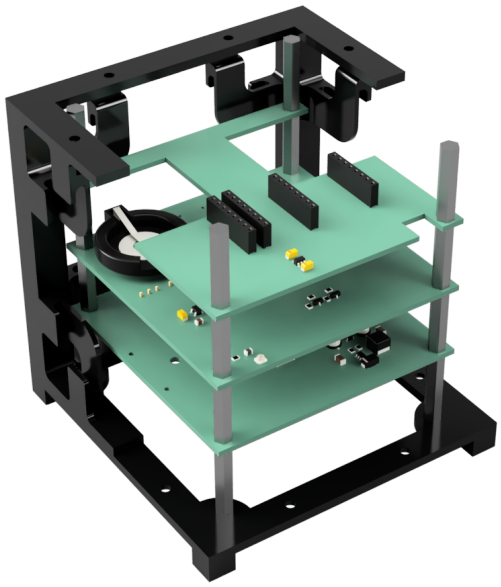

Rysunek 16: Wymiary elementu narożnego



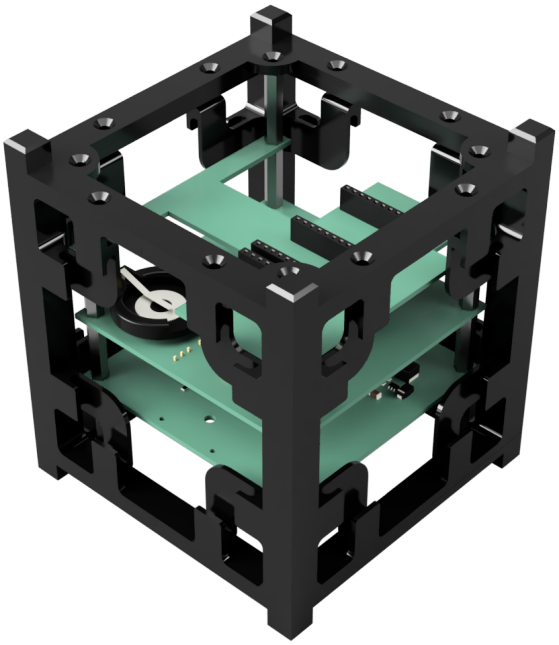
Rysunek 17: Obudowa podzielona na dwie cześci



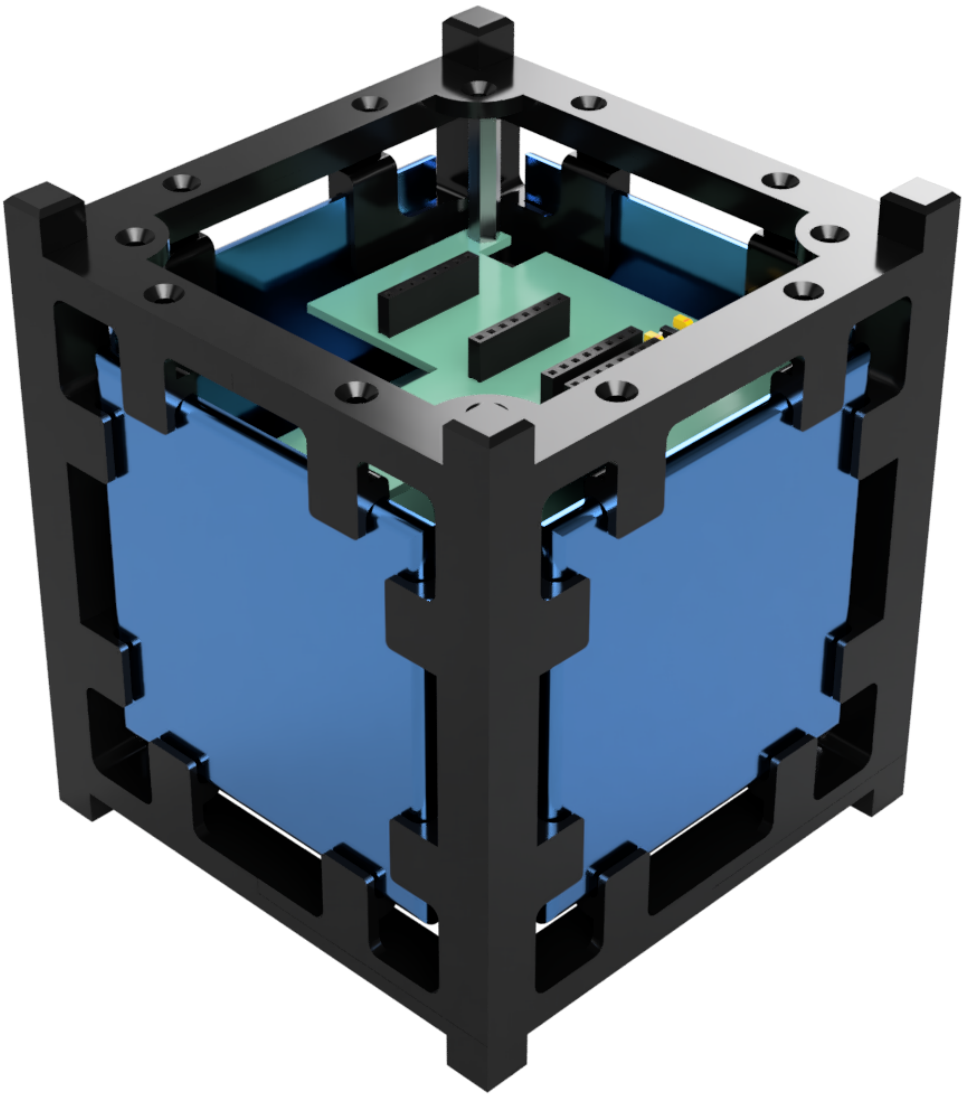
Rysunek 18: Obudowa zmontowana



Rysunek 19: Częściowo otwarta obudowa. Widoczne są elementy dystansowe oraz mocujące. Z odkrytej strony znajdują się gniazdo USB płytki Raspberry Pi, slot kart SD oraz gniazdo baterii zapasowej.



Rysunek 20: Obudowa kompletna bez ogniw



Rysunek 21: Obudowa z ogniwami fotowoltaicznymi

# Software

## 4.1 Architektura systemu

### 4.1.1 Uruchamianie systemu

Sekwencja startowa przebiega w następujący sposób:

1. Inicjalizacja sprzętowa mikrokontrolera Raspberry Pi Pico:
   (a) Konfiguracja standardowego wejścia/wyjścia poprzez `stdio_init_all()`
   (b) Inicjalizacja portów UART:
      - `uart0` - debugowanie i komunikacja
      - `uart1` - GPS
   (c) Konfiguracja pinów GPIO, w tym diody LED
   (d) Inicjalizacja magistral I2C:
      - `i2c0` - INA3221 oraz zegar czasu rzeczywistego
      - `i2c1` - czujniki środowiskowe
   (e) Włączenie zasilania GPS i czujników
   (f) Utworzenie instancji `SystemStateManager`
   (g) Emisja zdarzenia `GPSEvent::POWER_ON`
2. Inicjalizacja modułów peryferyjnych:
   (a) Inicjalizacja modułu radiowego i zapisanie statusu
   (b) Montowanie karty SD i weryfikacja statusu
   (c) Utworzenie pliku dziennika na karcie SD (jeśli dostępna)
   (d) Inicjalizacja czujników środowiskowych i światła
   (e) Zapisanie statusów inicjalizacji w menedżerze stanu systemu
   (f) Wysłanie ramki potwierdzającej rozpoczęcie pracy przez radio
3. Określenie trybu pracy systemu:

(a) Inicjalizacja i konfiguracja menedżera zasilania

(b) Odczyt napięcia baterii i poboru prądu

(c) Określenie źródła zasilania (USB lub bateria)

(d) Ustawienie odpowiedniego trybu pracy w `SystemStateManager`

4. Uruchomienie drugiego rdzenia:

   (a) Start funkcji `core1_entry()` na drugim rdzeniu procesora

   (b) Inicjalizacja menedżera telemetrii

   (c) Uruchomienie pętli obsługującej:

      - Zbieranie danych GPS
      - Zbieranie i zapisywanie danych telemetrycznych
      - Obsługę żądań resetu do trybu bootloadera

5. Finalizacja procesu uruchomienia:

   (a) Wygenerowanie i wysłanie komunikatu o trybie pracy

   (b) Zapisanie informacji o zakończeniu inicjalizacji z timestampem

   (c) Przejście do głównej pętli programu na pierwszym rdzeniu

## 4.2 Model komunikacji

W trakcie działania satelity konieczna jest komunikacja z systemem naziemnym w celu przesyłania danych telemetrycznych, konfiguracji oraz zarządzania systemem. Generowane i przetwarzane są duże ilości różnego rodzaju danych - sterujące działaniem systemu, zawierające informacje o stanie technicznym oraz pomiary z czujników.

Użytkownik nie powinien mieć możliwości wpływania na działanie systemu w sposób niezamierzony, dlatego zaimplementowałem prosty protokół komunikacyjny, który pozwala na odczyt i modyfikację tylko wybranych parametrów oraz wykonywanie określonych poleceń.

Odczytywanie danych oraz zmiana parametrów odbywa się za pomocą systemu komend które bezpośrednio operują na poszczególnych podsystemach. Zdecydowa-

łem się na to w celu zapewnienia pewnego poziomu izolacji między użytkownikiem a systemem, co zwiększa bezpieczeństwo i stabilność działania.

Każde polecenie ma zdefiniowane prawa dostępu, które określają dozwolone rodzaje operacji - odczyt, zapis lub odczyt i zapis. Proces przetwarzania ramki komunikacyjnej jest zabezpieczony przed błędami i zapewnia bezpieczne wykonanie operacji. Przebiega on w następujący sposób:

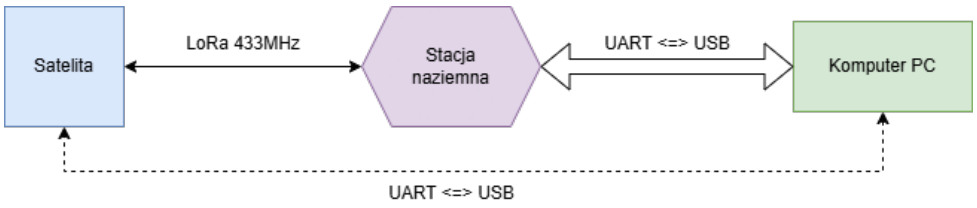
1. **Początkowy odbiór danych**
    - Główna pętla w `main.cpp` sprawdza czy moduł radiowy lub port szeregowy otrzymały dane
    - Po odebraniu pakietu LoRa wywoływana jest funkcja `on_receive(packet_size)`
    - Dla danych z portu szeregowego wywoływana jest funkcja `handle_uart_input()`
2. **Przetwarzanie odebranych danych LoRa w** `on_receive`
    - Odczyt surowych bajtów z modułu LoRa do bufora
    - Sprawdzenie rozmiaru pakietu (nieprzekraczający `MAX_PACKET_SIZE`)
    - Przekazanie bufora do funkcji `process_lora_packet()`
3. **Przetwarzanie metadanych pakietu w** `process_lora_packet`
    - Sprawdzenie rozmiaru pakietu (minimum 2 bajty)
    - Ekstrakcja i walidacja informacji adresowych (pierwsze 2 bajty)
    - Sprawdzenie czy adres docelowy zgadza się z adresem lokalnym
    - Sprawdzenie czy adres źródłowy zgadza się z oczekiwanym adresem zdalnym
    - Utworzenie ciągu znaków z pozostałych bajtów (właściwy ładunek)
    - Wyświetlenie zrzutu szesnastkowego dla celów debugowania
    - Przekazanie wyodrębnionego ciągu znaków do `extract_and_process_frames()`
4. **Przetwarzanie danych UART w** `handle_uart_input`
    - Odczyt znaków z portu szeregowego do bufora
    - Gdy napotkany zostanie znak końca linii (`\r` lub `\n`):

- Wyświetlenie odebranego ciągu znaków dla celów debugowania
- Przekazanie bufora do `extract_and_process_frames()`
- Wyczyszczenie bufora

5. **Ekstrakcja i przetwarzanie ramek w** `extract_and_process_frames`
   - Iteracyjne wyszukiwanie par znaczników `FRAME_BEGIN` ("KBST") i `FRAME_END` ("TSBK")
   - Dla każdej znalezionej pary:
     - Wyodrębnienie kompletnej ramki między znacznikami
     - Przekazanie wyodrębnionej ramki do `frame_process()` z odpowiednim typem interfejsu
   - Obsługa przypadku braku prawidłowych ramek w danych
6. **Przetwarzanie ramki w** `frame_process`
   - Dekodowanie ramki przy użyciu `frame_decode()`
   - Obliczenie klucza komendy na podstawie grupy i ID komendy
   - Wywołanie `execute_command()` z kluczem komendy, wartością i typem operacji
7. **Wykonanie komendy w** `execute_command`
   - Wyszukanie obsługi komendy w mapie `command_handlers`
   - Wywołanie odpowiedniej funkcji obsługi (np. `handle_list_files`)
   - Sprawdzenie poprawności rodzaju operacji (GET/SET)
   - Wygenerowanie odpowiedzi w postaci jednej lub więcej ramek
8. **Wysyłanie odpowiedzi**
   - Zakodowanie każdej ramki odpowiedzi przy użyciu `frame_encode()`
   - W przypadku interfejsu LoRa, przekazanie ramki przez radio
   - W przypadku interfejsu UART, wysłanie ramki przez port szeregowy

### 4.2.1 Interfejsy komunikacyjne

W modelu satelity dostępne są dwa interfejsy: lokalny - port UART oraz zdalny - moduł LoRa. Oba interfejsy obsługują żądania i odpowiedzi w formie ramek w identyczny sposób przy czym odpowiedź jest zwracana na interfejs który wysłał żądanie. Poniższy rysunek w uproszczeniu przedstawia schemat komunikacji w systemie:




Rysunek 22: Schemat komunikacji w systemie

#### Port szeregowy

W ramach implementacji systemu komunikacji, wykorzystałem interfejs UART jako podstawowy kanał do debugowania i kontroli satelity. Zaimplementowałem wielopoziomowy system logowania z następującymi poziomami szczegółowości:

- **ERROR** - krytyczne błędy systemu (czerwony kolor)
- **WARNING** - ostrzeżenia i ważne zdarzenia (żółty kolor)
- **INFO** - informacje o normalnej pracy (zielony kolor)
- **DEBUG** - szczegółowe informacje diagnostyczne (niebieski kolor)

Komunikacja przez UART została zabezpieczona mutexem, co zapewnia bezpieczny dostęp z obu rdzeni procesora. Każdy komunikat jest opatrzony znacznikiem czasu (w milisekundach od uruchomienia) oraz identyfikatorem rdzenia, z którego został wysłany.

Port szeregowy wykorzystywany do bezpośredniej interakcji z systemem, konfiguracji oraz monitorowania stanu systemu to port `uart0` pracujący z prędkością 115200 baud.

Zaimplementowałem również tryb przekierowania komunikacji GPS na port debugowania, co umożliwia bezpośrednią konfigurację modułu GPS poprzez oprogramowanie u-center - szczegółowy opis tego trybu znajduje się w sekcji4.10.5.

#### Moduł radiowy

Obsługę modułu radiowego LoRa zrealizowałem za pomocą biblioteki pico-lora[2], która dostarcza interfejs do obsługi modułu SX1278. Moduł komunikuje się z procesorem za pomocą interfejsu `spi0` z częstotliwością 8MHz, do którego ma ekskluzywny dostęp. Wysyłanie i odbieranie danych odbywa się w trybie half-duplex, co oznacza że moduł nie może odbierać danych podczas nadawania i odwrotnie.

### 4.2.2 Struktura ramki komunikacyjnej

Rozmiar ramki jest ograniczony przez możliwości modułu radiowego i wynosi maksymalnie 255 bajtów. Odliczając informacje dodatkowe (nagłówek, stopka, typ operacji, identyfikatory grupy i polecenia oraz znaki oddzielające) pozostaje około 235 bajtów na dane. W przypadku poleceń których złożoność przekracza ten limit, dane są przesyłane w formie sekwencji, gdzie każda ramka zawiera fragment danych.

Komendy są przesyłane w ustandaryzowanej formie ramki komunikacji a jej ogólna struktura jest opisana w tabeli 4.2.

#### Typy operacji

| Typ operacji | Opis |
|---|---|
| GET | Żądanie odczytu danych, wysyłane przez stację naziemną |
| SET | Żądanie ustawienia wartości parametru |
| RES | Potwierdzenie wykonania operacji |
| VAL | Przesłanie wartości (odpowiedź na GET) |
| SEQ | Element sekwencji danych (np. przy przesyłaniu logów) |
| ERR | Sygnalizacja błędu wykonania operacji |

Tabela 4.1: Typy operacji

| Element | Format | Opis |
|---|---|---|
| Nagłówek | KBST | Stały znacznik początku ramki |
| Kierunek | 0/1 | 0: stacja→satelita, 1: satelita→stacja |
| Typ operacji | GET/SET<br>RES/VAL/ERR/SEQ | Rodzaj operacji w ramce |
| Grupa | 0-10 | Identyfikator podsystemu (np. zasilanie, GPS) |
| Polecenie | 0-10 | Identyfikator konkretnej komendy w grupie |
| Wartość | string | Parametry lub dane odpowiedzi |
| Jednostka | string | Jednostka wartości (np. V, mA, s) (opcjonalnie) |
| Stopka | TSBK | Stały znacznik końca ramki |

Tabela 4.2: Struktura ramki komunikacyjnej

Poszczególne pola są oddzielone znakiem średnika (;), co ułatwia parsowanie i debugowanie.

**Grupy komend**

| Grupa | Opis |
|---|---|
| 1 | Diagnostyka systemu |
| 3 | Zarządzanie zegarem |
| 5 | System zdarzeń |
| 7 | Obsługa GPS |
| 8 | System telemetrii |

Tabela 4.3: Grupy komend

Przykładowe ramki dla różnych typów operacji:

**KBST;0;GET;1;1;;TSBK**

Pobranie wartości parametru z grupy 1, polecenie 1 - wersja oprogramowania. W przypadku operacji GET pole wartości jest puste.

**KBST;0;SET;7;1;1;TSBK**

Zapis wartości 1 do parametru z grupy 7, polecenie 1 - sterowanie zasilaniem GPS (0 - wyłączony, 1 - włączony). W przypadku operacji SET pole wartości zawiera wartość do zapisu.

**KBST;1;RES;1;8;3;TSBK**

Potwierdzenie wykonania operacji SET w grupie 1, polecenie 5 - zmiana poziomu logowania wiadomości UART. Pole wartości zawiera wartość w postaci tekstowej oznaczający wynik operacji.

**KBST;1;VAL;3;4;25.25;C;TSBK**

Odpowiedź na żądanie odczytu wartości z grupy 3, polecenie 4 - temperatura zmierzona przez układ zegara. Pole wartości zawiera odczytaną wartość (25.25), a pole jednostki zawiera 'C'.

**KBST;1;SEQ;1;0;1.0-1.1-1.2-1.3-1.8-1.9-3.0-3.1-3.4-5.1-5.2-7.1-7.2-8.2-8.3;TSBK**

**KBST;1;VAL;1;0;SEQ_DONE;TSBK**

Element sekwencji danych - odpowiedź na żądanie odczytu danych z grupy 1, polecenie 0 - odczyt informacji o dostępności poleceń w aktualnie pracującej wersji programu. Pole wartości zawiera sekwencję grupa-polecenie oddzielone kropkami. W przypadku gdy dostępnych poleceń byłoby więcej, sekwencja zostałaby podzielona na kilka ramek. Ostatnia ramka zawiera informację o zakończeniu sekwencji.

**KBST;1;ERR;3;0;INVALID_FORMAT;;TSBK**

Sygnalizacja błędu w przetwarzaniu polecenia w grupie 3, polecenie 0 - ustawienie czasu. Pole wartości zawiera kod błędu 'INVALID_FORMAT' informujący o przyczynie błędu - nieprawidłowy format danych.

Taka struktura ramek zapewnia jednoznaczną identyfikację początku i końca ramki, łatwą rozszerzalność protokołu o nowe komendy, efektywne wykrywanie i obsługę błędów oraz możliwość przesyłania danych w różnych formatach i jednostkach.

## 4.3 Menadżer stanu systemu

### 4.3.1 Wprowadzenie

W ramach implementacji systemu zarządzania stanem całego systemu stworzyłem klasę SystemStateManager, która wykorzystuje wzorzec projektowy Singleton. Klasa ta służy jako centralny punkt do przechowywania i zarządzania różnymi stanami systemowymi satelity.

### 4.3.2 Zarządzanie stanem

Zaimplementowałem następujące stany systemowe:

```
// Flaga oznaczająca oczekujące żądanie ponownego uruchomienia systemu w trybie
    wymiany oprogramowania
bool pending_bootloader_reset;
// Flaga oznaczająca żadanie wstrzymania zbierania danych GPS
bool gps_collection_paused;
// Flaga oznaczająca stan zamontowania karty SD
bool sd_card_mounted;
// Poziom szczegółowości logów UART
VerbosityLevel uart_verbosity;
// Flaga oznaczają stan inicjacjizacji karty SD
bool sd_card_init_status;
// Flaga oznaczają stan inicjalizacji radia
bool radio_init_status;
// Flaga oznaczają stan inicjalizacji czujnika poziomu oświetlenia
bool light_sensor_init_status;
// Flaga informująca o stanie inicjalizacji czujnika środowiskowego
bool env_sensor_init_status;
// Mutex dla bezpiecznego dostępu do zmiennych stanu
recursive_mutex_t mutex_;
```

Listing 4.1: Stany systemowe

### 4.3.3 Tryb pracy satelity

Przy starcie systemu, układ pomiarowy na podstawie wartości napięć baterii i zasilania oraz pobieranego przez system prądu określa czy satelita jest w trakcie lotu (pracuje na własnym zasilaniu) czy w trybie naziemnym (zasilany z USB).

W trybie naziemnym wszystkie polecenia są aktywne i dozwolone jest ich wykonywanie. W trybie lotu niektóre polecenia np. wyłączenie zasilania GPS są dezaktywowane aby zapobiec niepożądanemu działaniu systemu.

### 4.3.4 Komendy diagnostyczne

System diagnostyczny udostępnia następujące komendy służące do monitorowania i kontroli stanu systemu:

| ID | Format komendy | Opis |
|---|---|---|
| 1.0 | KBST;0;GET;1;0;;TSBK | Wyświetla listę wszystkich dostępnych komend systemowych wraz z ich identyfikatorami. |
| 1.1 | KBST;0;GET;1;1;;TSBK | Pobiera numer wersji oprogramowania układowego (firmware). |
| 1.2 | KBST;0;GET;1;2;;TSBK | Pobiera aktualny tryb pracy systemu (USB/akumulatory). |
| 1.3 | KBST;0;GET;1;3;;TSBK | Pobiera czas pracy w sekundach od uruchomienia. |
| 1.8 | KBST;0;GET;1;8;;TSBK | Pobiera aktualny poziom szczegółowości logów UART (0-4). |
| 1.8 | KBST;0;SET;1;8;[poziom];TSBK | Ustawia nowy poziom szczegółowości logów UART (0-4). |
| 1.9 | KBST;0;SET;1;9;USB;TSBK | Uruchamia system ponownie w trybie bootloadera USB (tryb programowania). |

Tabela 4.4: Komendy diagnostyczne systemu

## 4.4 Menadżer zasilania

### 4.4.1 Wprowadzenie

Klasa `PowerManager` stanowi centralny punkt interakcji z układem INA3221. Jej głównym zadaniem jest zapewnienie abstrakcji sprzętowej, umożliwiając łatwy odczyt danych pomiarowych oraz konfigurację parametrów pracy układu. W ramach tej klasy zaimplementowałem szereg funkcji, które obejmują inicjalizację układu INA3221, odczyt jego identyfikatorów, pomiar napięć i prądów na poszczególnych kanałach oraz konfigurację trybów pracy.

### 4.4.2 Architektura systemu

Inicjalizacja układu odbywa się przez wywołanie metody `INA3221::begin()`. Metoda ta sprawdza poprawność połączenia z układem oraz weryfikuje jego identyfikatory, aby upewnić się, że komunikacja przebiega prawidłowo. Odczyt identyfikatorów urządzenia, takich jak identyfikator producenta (`INA3221::get_manufacturer_id()`) oraz identyfikator wersji (`INA3221::get_die_id()`), pozwala na jednoznaczną identyfikację układu i weryfikację jego autentyczności.

Konstruktor klasy `PowerManager` przyjmuje jako argument wskaźnik na instancję `i2c_inst_t`, co daje dużą elastyczność w wyborze portu I2C, do którego podłączony jest układ INA3221. Dodatkowo, w konstruktorze inicjalizowany jest mutex o nazwie `powerman_mutex_`. Mutex ten pełni kluczową rolę w zapewnieniu bezpiecznego dostępu do zasobów klasy z różnych rdzeni procesora, co jest szczególnie istotne w systemach wielowątkowych, gdzie równoczesny dostęp do danych pomiarowych mógłby prowadzić do niepożądanych konfliktów i błędów.

```
1  class PowerManager {
2  private:
       INA3221 ina3221_;
       bool initialized_;
       recursive_mutex_t powerman_mutex_;
```

```
7       static constexpr float SOLAR_CURRENT_THRESHOLD = 50.0f;
        static constexpr float USB_CURRENT_THRESHOLD = 50.0f;
        static constexpr float BATTERY_LOW_THRESHOLD = 2.8f;
        static constexpr float BATTERY_FULL_THRESHOLD = 4.2f;
```

Listing 4.2: Klasa PowerManager

### 4.4.3 Konfiguracja INA3221

Po pomyślnej inicjalizacji układu INA3221, następuje proces jego konfiguracji. W ramach tego procesu ustawiane są progi alarmowe dla napięcia baterii, definiując zakresy wartości, które uznawane są za normalne, zbyt niskie lub zbyt wysokie. Progi te, zdefiniowane jako `VOLTAGE_LOW_THRESHOLD` oraz `VOLTAGE_OVERCHARGE_THRESHOLD`, pozwalają na wczesne wykrywanie potencjalnych problemów z zasilaniem.

Do konfiguracji układu służy metoda `configure`, która przyjmuje jako argument mapę konfiguracji. Mapa ta zawiera pary klucz-wartość, gdzie kluczem jest nazwa parametru do skonfigurowania, a wartością jest jego nowa wartość. Metoda `configure` pozwala na zmianę trybu pracy układu (ciągły, wyzwalany) oraz trybu uśredniania próbek. Dostępne tryby uśredniania pozwalają na redukcję szumów i zwiększenie dokładności pomiarów, kosztem wydłużenia czasu konwersji.

### 4.4.4 Pomiary

Klasa `PowerManager` udostępnia szereg metod, które umożliwiają odczyt kluczowych parametrów systemu zasilania. Metody te obejmują pomiary napięć: baterii oraz wyjścia przetwornicy 5V, a także prądów: ładowania - zarówno z paneli fotowoltaicznych jak i z USB - oraz rozładowania.

Wszystkie metody odczytu są zabezpieczone mutexem `powerman_mutex_`, co gwarantuje spójność danych i zapobiega konfliktom w przypadku równoczesnego dostępu z różnych rdzeni procesora. Z metod tych korzysta system telemetrii do przesyłania danych o stanie zasilania do stacji naziemnej oraz logowania ich na karcie SD.

## 4.5 System zdarzeń

System zdarzeń jest kluczowym elementem oprogramowania satelity, pozwalającym na śledzenie wszystkich istotnych procesów zachodzących w systemie. Głównym celem systemu zarządzania zdarzeniami jest umożliwienie:

- Rejestrowania wszystkich istotnych zdarzeń w systemie
- Przechowywania historii zdarzeń w pamięci nieulotnej (na karcie SD)
- Szybkiego dostępu do ostatnich zdarzeń nawet w przypadku braku dostępu do pamięci trwałej
- Kategoryzacji zdarzeń według grup i typów
- Zdalnego dostępu do zarejestrowanych zdarzeń poprzez system komunikacji

### 4.5.1 Architektura systemu zarządzania zdarzeniami

#### Struktura klas

System zarządzania zdarzeniami składa się z następujących głównych komponentów:

1. **Klasa EventManager** - singleton zarządzający buforem zdarzeń i operacjami I/O
2. **Klasa EventEmitter** - interfejs umożliwiający emisję zdarzeń z dowolnego miejsca w kodzie
3. **Struktura EventLog** - przechowująca dane pojedynczego zdarzenia
4. **Enumy EventGroup i specyficzne dla grup** - definiujące kategorie i typy zdarzeń

#### Struktura EventLog

Dla efektywnego przechowywania zdarzeń zaprojektowałem strukturę EventLog, która zawiera następujące pola:

```
class EventLog {
    public:
        uint32_t timestamp;  // znacznik czasu w formacie Unix
        uint16_t id;         // unikalne ID zdarzenia
        uint8_t group;       // grupa zdarzeń (np. SYSTEM, POWER)
        uint8_t event;       // konkretny typ zdarzenia
} __attribute__((packed));
```

Listing 4.3: Struktura EventLog

Struktura ta jest zoptymalizowana pod kątem rozmiaru - zajmuje tylko 8 bajtów w pamięci, co umożliwia efektywne przechowywanie dużej liczby zdarzeń nawet przy ograniczonych zasobach pamięciowych mikrokontrolera RP2040.

### Klasa EventManager

Sercem całego systemu jest klasa EventManager, którą zaimplementowałem zgodnie z wzorcem projektowym Singleton. Zapewnia to jeden punkt dostępu do funkcji zarządzania zdarzeniami w całym systemie. Jej implementacja wygląda następująco:

```
class EventManager {
private:
    EventLog events[EVENT_BUFFER_SIZE]; // bufor cykliczny zdarzeń
    size_t eventCount;                  // liczba zdarzeń w buforze
    size_t writeIndex;                  // indeks do zapisu nowego zdarzenia
    mutex_t eventMutex;                 // mutex do synchronizacji wątków
    uint16_t nextEventId;               // inkrementalne ID dla zdarzeń
    size_t eventsSinceFlush;            // liczba zdarzeń od ostatniego zapisu na
     SD

    // prywatny konstruktor - wzorzec Singleton
    EventManager();

public:
    static EventManager& get_instance();
```

```
    bool init();
    void log_event(uint8_t group, uint8_t event);
    const EventLog& get_event(size_t index) const;
    size_t get_event_count() const;
    bool save_to_storage();
};
```

Listing 4.4: Klasa EventManager

### Grupy i typy zdarzeń

Przy projektowaniu systemu określiłem hierarchiczną strukturę zdarzeń, co umożliwia ich efektywną kategoryzację i filtrowanie. Zdarzenia są podzielone na grupy, a w ramach każdej grupy wyróżnione są konkretne typy zdarzeń:

```
enum class EventGroup : uint8_t {
    SYSTEM = 0x00,
    POWER = 0x01,
    COMMS = 0x02,
    GPS = 0x03,
    CLOCK = 0x04
};

enum class SystemEvent : uint8_t {
    BOOT = 0x01,
    SHUTDOWN = 0x02,
    WATCHDOG_RESET = 0x03,
    CORE1_START = 0x04,
    CORE1_STOP = 0x05
};

// Podobne enumy istnieją dla pozostałych grup
```

Listing 4.5: Przykładowe enumeracje zdarzeń

Ta hierarchiczna struktura pozwala mi na łatwe dodawanie nowych grup i typów

zdarzeń w przyszłości, bez konieczności znaczących zmian w istniejącym kodzie.

### 4.5.2 Generowanie zdarzeń

Dzięki zastosowaniu klasy EventEmitter, każdy moduł systemu może generować zdarzenia w prosty sposób - wystarczy wywołać funkcję `emit` z odpowiednimi parametrami, na przykład:

```
1    EventEmitter::emit(EventGroup::SYSTEM, SystemEvent::BOOT);
```

Listing 4.6: Funkcja emitująca zdarzenie

Wywołanie tej funkcji spowoduje zarejestrowanie zdarzenia z grupy SYSTEM i typie BOOT w buforze zdarzeń przez metodę `log_event` klasy EventManager.

```
1  class EventEmitter {
   public:
       /**
4       * @brief Emits an event.
        * @param[in] group Event group.
        * @param[in] event Event code.
        * @tparam T Type of the event enumeration.
        */
9      template<typename T>
       static void emit(EventGroup group, T event) {
           EventManager::get_instance().log_event(
               static_cast<uint8_t>(group),
               static_cast<uint8_t>(event)
14         );
       }
   };
```

Listing 4.7: Klasa EventEmitter

### 4.5.3 Rejestracja zdarzeń

Centralnym punktem systemu jest funkcja `log_event`. Odpowiada ona za zapisywanie zdarzeń w buforze cyklicznym, aktualizację indeksów, logowanie do UART oraz automatyczne zapisywanie zdarzeń do pamięci nieulotnej. Zapis do pamięci nieulotnej odbywa się wtedy, gdy osiągnięto próg zdarzeń lub gdy zdarzenie jest krytyczne (np. związane z zasilaniem).

```
void EventManager::log_event(uint8_t group, uint8_t event) {
    // Blokowanie mutexu, aby zapewnić atomowość operacji
    mutex_enter_blocking(&eventMutex);

    // Pobieranie znacznika czasu z zegara RTC
    uint32_t timestamp = DS3231::get_instance().get_local_time();
    uint16_t id = nextEventId++;

    // Zapisanie zdarzenia w buforze cyklicznym
    EventLog& log = events[writeIndex];
    log.id = id;
    log.timestamp = timestamp;
    log.group = group;
    log.event = event;

    // Aktualizacja indeksów i liczników
    writeIndex = (writeIndex + 1) % EVENT_BUFFER_SIZE;
    if (eventCount < EVENT_BUFFER_SIZE) {
        eventCount++;
    }
    eventsSinceFlush++;

    // Zwolnienie mutexu
    mutex_exit(&eventMutex);

    // Logowanie do UART dla celów diagnostycznych
    std::string event_string = "Event: " + std::to_string(id) +
```

```
        " Group: " + std::to_string(group) +
        " Event: " + std::to_string(event);
    uart_print(event_string, VerbosityLevel::WARNING);

    // Automatyczny zapis do pamięci trwałej jeśli:
    // - osiągnięto próg zdarzeń lub
    // - zdarzenie dotyczy zasilania (krytyczne)
    if (eventsSinceFlush >= EVENT_FLUSH_THRESHOLD ||
        group == static_cast<uint8_t>(EventGroup::POWER)) {
        save_to_storage();
        eventsSinceFlush = 0;
    }
}
```

Listing 4.8: Implementacja funkcji log_event

### 4.5.4 Dostęp do zdarzeń

Dla dostępu do zarejestrowanych zdarzeń zaimplementowałem funkcję `get_event`. Jest ona używana przez system poleceń do zdalnego odczytu logu zdarzeń.

```
const EventLog& EventManager::get_event(size_t index) const {
    mutex_enter_blocking(const_cast<mutex_t*>(&eventMutex));

    // Sprawdzenie poprawności indeksu
    if (index >= eventCount) {
        static EventLog emptyEvent;
        mutex_exit(const_cast<mutex_t*>(&eventMutex));
        return emptyEvent;
    }

    // Obliczenie rzeczywistego indeksu w buforze cyklicznym
    size_t readIndex;
    if (eventCount == EVENT_BUFFER_SIZE) {
        readIndex = (writeIndex + index) % EVENT_BUFFER_SIZE;
```

```
    }
    else {
        readIndex = index;
    }

    const EventLog& event = events[readIndex];
    mutex_exit(const_cast<mutex_t*>(&eventMutex));
    return event;
}
```

Listing 4.9: Implementacja funkcji get_event

### 4.5.5 Polecenia systemu zdarzeń

Z systemem zdarzeń są związane następujące polecenia:

| ID | Format komendy | Opis |
|---|---|---|
| 5.1 | KBST;0;GET;5;1;[N];TSBK | Pobiera ostatnie N zdarzeń z logu zdarzeń. Jeśli N=0, pobiera wszystkie zdarzenia. |
| 5.2 | KBST;0;GET;5;2;;TSBK | Pobiera całkowitą liczbę zdarzeń w logu. |

Tabela 4.5: Komendy systemu zdarzeń

## 4.6 System telemetrii

### 4.6.1 Wprowadzenie

System telemetrii jest jednym z kluczowych podsystemów, który zaprojektowałem, aby umożliwić monitoring stanu urządzenia podczas jego pracy. W niniejszym rozdziale szczegółowo omówię architekturę systemu, jego implementację oraz zasadę działania.

System telemetrii odpowiada za zbieranie, przechowywanie i udostępnianie danych z różnych czujników i podsystemów satelity, takich jak: podsystem zasilania, czujniki środowiskowe oraz moduł GPS. Gromadzenie tych danych pozwala na bieżący

monitoring parametrów satelity oraz umożliwia wykrywanie potencjalnych problemów zanim staną się krytyczne.

### 4.6.2 Architektura systemu telemetrii

System telemetrii zaimplementowałem w oparciu o wzorzec projektowy Singleton, co zapewnia istnienie tylko jednej instancji menedżera telemetrii w całym systemie. Jest to szczególnie istotne, ponieważ dostęp do systemu telemetrii jest wymagany z różnych miejsc w kodzie, a jednocześnie musimy zapewnić spójność gromadzonych danych.

Centralnym elementem systemu jest klasa `TelemetryManager`, która udostępnia interfejs do zbierania i zarządzania danymi telemetrycznymi. Dostęp do tej klasy można uzyskać poprzez metodę statyczną `get_instance()`.

Dane telemetryczne przechowywane są w dwóch strukturach: `TelemetryRecord` przechowuje informacje o napięciach, prądach oraz dane z GPS, oraz `SensorDataRecord` która przechowuje odczyty z czujników środowiskowych. Obie struktury wyposażyłem w metodę `to_csv()`, która konwertuje dane na format CSV, co ułatwia zapisywanie rekordów telemetrycznych do plików oraz ich przesyłanie za pomocą systemu komunikacji.

Inicjalizacja systemu telemetrii jest przeprowadzana przez metodę `init()`, która wykonuje następujące kroki:

1. Inicjalizacja muteksu do synchronizacji dostępu do buforów
2. Sprawdzenie, czy karta SD jest zamontowana
3. Utworzenie plików CSV z nagłówkami, jeśli nie istnieją, w przeciwnym razie wyczyszczenie zawartości istniejących plików

| Pole | Opis |
|---|---|
| timestamp | Znacznik czasu (Unix timestamp) |
| build_version | Wersja oprogramowania |
| battery_voltage | Napięcie baterii (V) |
| system_voltage | Napięcie systemowe 5V (V) |
| charge_current_usb | Prąd ładowania z USB (mA) |
| charge_current_solar | Prąd ładowania z paneli słonecznych (mA) |
| discharge_current | Prąd rozładowania baterii (mA) |
| time | Czas UTC (GPS RMC) |
| latitude | Szerokość geograficzna (GPS RMC) |
| lat_dir | Kierunek szerokości geograficznej (N/S, GPS RMC) |
| longitude | Długość geograficzna (GPS RMC) |
| lon_dir | Kierunek długości geograficznej (E/W, GPS RMC) |
| speed | Prędkość (w węzłach, GPS RMC) |
| course | Kurs (w stopniach, GPS RMC) |
| date | Data (GPS RMC) |
| fix_quality | Jakość fixa GPS (GPS GGA) |
| satellites | Liczba satelitów (GPS GGA) |
| altitude | Wysokość (m, GPS GGA) |

Tabela 4.6: Zawartość struktury `TelemetryRecord`

| Pole | Opis |
|---|---|
| timestamp | Znacznik czasu |
| temperature | Temperatura |
| pressure | Ciśnienie |
| humidity | Wilgotność |
| light | Natężenie światła |

Tabela 4.7: Zawartość struktury `SensorDataRecord`

### 4.6.3 Zbieranie danych telemetrycznych

Zbieranie danych telemetrycznych podzieliłem na kilka specjalizowanych funkcji, co pozwala na lepszą organizację kodu oraz jego łatwiejszą konserwację. Główną funkcją odpowiedzialną za zbieranie danych jest `collect_telemetry()`, która koordynuje proces pozyskiwania informacji z różnych podsystemów.

Proces zbierania danych rozpoczyna się od pobrania aktualnego czasu z modułu RTC (DS3231), który służy jako znacznik czasowy dla rekordów telemetrycznych. Następnie kolejno zbierane są dane z poszczególnych podsystemów: dane o stanie zasilania pozyskiwane są za pomocą metody `collect_power_telemetry()`, która korzysta z instancji klasy `PowerManager` do odczytu aktualnych wartości napięć i prądów. Na podstawie tych danych generowane są również zdarzenia systemowe, które mogą być wykorzystane przez inne podsystemy do reagowania na zmiany w stanie zasilania.

Na podstawie zebranych danych o zasilaniu, system telemetrii generuje zdarzenia informujące o istotnych zmianach stanu, takich jak podłączenie zasilania USB, rozpoczęcie ładowania z paneli słonecznych czy niski poziom naładowania baterii. Funkcja `emit_power_events()` analizuje aktualne parametry i emituje odpowiednie zdarzenia, które są następnie przetwarzane przez system zarządzania zdarzeniami satelity.

Dane z modułu GPS pobierane są za pomocą funkcji `collect_gps_telemetry()`, która odczytuje przetworzone ramki NMEA z klasy `NMEAData`. System analizuje zarówno ramki RMC (Recommended Minimum Navigation Information), które zawierają podstawowe informacje nawigacyjne, jak i ramki GGA (Global Positioning System Fix Data), które dostarczają dodatkowych informacji o jakości fixa GPS i wysokości.

Dane środowiskowe są zbierane przez funkcję `collect_sensor_telemetry()`, która korzysta z klasy `SensorWrapper` do komunikacji z czujnikami. System odczytuje wartości temperatury, ciśnienia, wilgotności oraz natężenia światła i zapisuje je w strukturze `SensorDataRecord`.

Zebrane dane telemetryczne są przechowywane w buforach cyklicznych, co pozwala na efektywne zarządzanie pamięcią. System wykorzystuje dwa bufory: jeden dla danych `TelemetryRecord` i drugi dla danych `SensorDataRecord`. Zaimplementowany mechanizm buforowania pozwala na przechowywanie określonej liczby najnowszych rekordów (domyślnie 20), która jest definiowana przez stałą `TELEMETRY_BUFFER_SIZE`.

Bufor cykliczny to efektywna struktura danych, która działa jak kolejka o stałym rozmiarze. Gdy bufor jest pełny, najstarsze dane są nadpisywane przez nowe. W implementacji systemu telemetrii wykorzystałem tablice o stałym rozmiarze wraz z indeksami wskazującymi na pozycje zapisu i licznikiem przechowywanych rekordów.

Ponieważ dane telemetryczne mogą być zbierane i odczytywane z różnych wątków, konieczne było zastosowanie mechanizmów synchronizacji. W tym celu wykorzystałem muteksy z biblioteki Raspberry Pi Pico, które zapewniają bezpieczny dostęp do buforów. Wszystkie operacje na buforach są chronione przez `telemetry_mutex`, co zapobiega konfliktom dostępu i zapewnia spójność danych.

Bufory telemetryczne są okresowo zapisywane na karcie SD, co pozwala na przechowywanie danych w przypadku braku zasilania oraz ich odzyskanie po zakończeniu lotu. Za zapisywanie danych odpowiada funkcja `flush_telemetry()`, która zapisuje wszystkie rekordy z buforów do plików CSV. System tworzy dwa pliki: `/telemetry.csv` dla głównych danych telemetrycznych oraz `/sensors.csv` dla danych z czujników.

System telemetrii działa według ustalonego harmonogramu, który określa częstotliwość zbierania danych i ich zapisywania na karcie SD. Harmonogram ten jest kontrolowany przez dwie kluczowe funkcje: `is_telemetry_collection_time()` oraz `is_telemetry_flush_time()`.

Dane telemetryczne są zbierane w regularnych odstępach czasu, które domyślnie wynoszą 1 sekundę (zdefiniowane przez `DEFAULT_SAMPLE_INTERVAL_MS`). Funkcja `is_telemetry_collection_time()` sprawdza, czy minął wymagany czas od ostatniego pomiaru i aktualizuje znacznik czasowy ostatniego zbierania danych.

Zebrane dane są zapisywane na karcie SD po zgromadzeniu określonej liczby rekordów, domyślnie 10 (zdefiniowane przez stałą `DEFAULT_FLUSH_THRESHOLD`). Funkcja `is_telemetry_flush_time()` sprawdza, czy licznik zebranych rekordów osiągnął próg zapisywania i resetuje licznik, jeśli jest to konieczne.

Opisane mechanizmy zbierania i zapisywania danych są zintegrowane z główną pętlą drugiego rdzenia mikrokontrolera (Core 1). W każdej iteracji pętli sprawdzany jest czas zbierania danych, a po zgromadzeniu odpowiedniej liczby rekordów, są one zapisywane na karcie SD.

### 4.6.4 Dostęp do danych telemetrycznych przez system komunikacji

System telemetrii udostępnia zebrane dane za pośrednictwem interfejsu komunikacyjnego satelity. Zaimplementowałem dwie komendy, które pozwalają na pobranie ostatnich zarejestrowanych rekordów telemetrycznych i czujnikowych.

Funkcja `handle_get_last_telemetry_record()` obsługuje komendę pobierania ostatniego rekordu telemetrycznego. Funkcja ta pobiera ostatni rekord z menedżera telemetrii, konwertuje go na format CSV i zwraca jako ramkę komunikacyjną.

Podobnie działa funkcja `handle_get_last_sensor_record()`, która obsługuje komendę pobierania ostatniego rekordu czujnikowego. Funkcja ta pobiera ostatni rekord danych z czujników i zwraca go jako ramkę komunikacyjną.

### 4.6.5 Komendy systemu telemetrii

System telemetrii udostępnia następujące komendy, które mogą być wykorzystane do interakcji z danymi telemetrycznymi:

| ID | Format komendy | Opis |
|---|---|---|
| 8.2 | KBST;0;GET;8;2;;TSBK | Pobiera ostatni rekord telemetryczny w formacie CSV. Zwraca dane o zasilaniu, położeniu GPS i inne parametry systemowe. |
| 8.3 | KBST;0;GET;8;3;;TSBK | Pobiera ostatni rekord danych z czujników w formacie CSV. Zwraca wartości temperatury, ciśnienia, wilgotności i natężenia światła. |

Tabela 4.8: Komendy systemu telemetrii

## 4.7 Implementacja systemu plików

### 4.7.1 Architektura systemu

System plików został zaimplementowany w sposób warstwowy, gdzie główne komponenty to:

- Warstwa urządzeń blokowych (obsługa karty SD)
- Warstwa systemu plików (FAT)
- Warstwa abstrakcji VFS (pico-vfs)

W ramach implementacji systemu plików dla KubiSat zdecydowałem się wykorzystać bibliotekę pico-vfs[7], Bazuje ona na architekturze wirtualnego systemu plików zaprojektowanej specjalnie dla Raspberry Pi Pico. Zapewnia ona ujednolicony interfejs do operacji plikowych poprzez standardowe API POSIX oraz funkcje standardowe języka C, umożliwiając tym samym efektywne i proste wykorzystanie różnych nośników pamięci, takich jak wbudowana pamięć flash oraz karty SD.

Modularna architektura pico-vfs pozwala na wymianę i dodawanie poszczególnych komponentów, dzięki czemu system jest elastyczny i skalowalny. Do jej użycia wymagane jest wywołanie funkcji `fs_init` z odpowiednimi parametrami, które tworzą instancje systemu plików i urządzenia blokowego; w przypadku karty SD jest to `blockdevice_sd_create`.

```
bool fs_init(void) {
    blockdevice_t *sd = blockdevice_sd_create(SD_SPI_PORT,
                                              SD_MOSI_PIN,
                                              SD_MISO_PIN,
                                              SD_SCK_PIN,
                                              SD_CS_PIN,
                                              24 * MHZ,
                                              false);
    filesystem_t *fat = filesystem_fat_create();
```

```
    int err = fs_mount("/", fat, sd);
    if (err == -1) {
        err = fs_format(fat, sd);
        if (err == -1) {
            return false;
        }
        err = fs_mount("/", fat, sd);
    }
    return err != -1;
}
```

Listing 4.10: Inicjalizacja systemu plików

Powyższy kod inicjalizuje system plików, montuje system plików FAT na karcie SD i formatuje go w przypadku braku dostępu.

### 4.7.2 Monitorowanie stanu

Stan systemu plików jest monitorowany poprzez klasę SystemStateManager: 4.3.2.

```
SystemStateManager::get_instance()
    .set_sd_card_mounted(true);
```

Listing 4.11: Pobieranie stanu karty SD

Każda operacja zamontowania/odmontowania systemu plików karty SD jest automatycznie odzwierciedlana w mendadżerze stanu. Magazyn danych jest wykorzystywany przez system telemetrii oraz system zdarzeń do zapisywania ich stanu.

## 4.8 System czujników

### 4.8.1 Wprowadzenie

Moduł czujników stanowi kluczowy podsystem, który zaprojektowałem w celu monitorowania warunków środowiskowych zarówno wewnątrz samego satelity, jak i w jego

otoczeniu.

Głównym celem systemu czujników jest dostarczanie aktualnych danych o parametrach fizycznych, które są istotne z punktu widzenia misji satelity oraz monitorowania jego stanu. System zaprojektowałem z myślą o modularności i rozszerzalności, co pozwala na łatwe dodawanie nowych czujników w przyszłości bez konieczności modyfikacji całej architektury.

### 4.8.2 Architektura systemu czujników

System czujników opiera się na interfejsie abstrakcyjnym `ISensor`, który zapewnia jednolity sposób interakcji z różnymi typami czujników, niezależnie od ich producenta czy modelu. Ta abstrakcja pozwala na łatwą wymianę czujników i upraszcza kod obsługujący dane pomiarowe.

Architektura systemu czujników składa się z kilku kluczowych elementów:

- Interfejs `ISensor` - definiuje wspólny interfejs dla wszystkich czujników
- Klasa `SensorWrapper` - zarządza wszystkimi czujnikami w systemie
- Implementacje konkretnych czujników (np. `BME280`, `BH1750`)
- Klasy opakowujące (wrappery) dla czujników, implementujące interfejs `ISensor`

#### Interfejs ISensor

Sercem systemu czujników jest interfejs `ISensor`, który zdefiniowałem jako abstrakcyjną klasę bazową dla wszystkich czujników w satelicie. Interfejs ten zapewnia jednolity sposób inicjalizacji czujników, konfiguracji ich parametrów oraz odczytywania danych pomiarowych. Dzięki temu reszta systemu może korzystać z czujników bez znajomości ich szczegółów implementacyjnych.

```
class ISensor {
public:
    virtual ~ISensor() = default;
    virtual bool init() = 0;
```

```
    virtual float read_data(SensorDataTypeIdentifier type) = 0;
    virtual bool is_initialized() const = 0;
    virtual SensorType get_type() const = 0;
    virtual bool configure(const std::map<std::string, std::string>& config) =
    0;
    virtual uint8_t get_address() const = 0;
};
```

Listing 4.12: Definicja interfejsu ISensor

Interfejs `ISensor` definiuje następujące metody:

- `init()` - inicjalizuje czujnik i przygotowuje go do pracy
- `read_data()` - odczytuje dane z czujnika dla określonego typu pomiaru
- `is_initialized()` - sprawdza, czy czujnik został poprawnie zainicjalizowany
- `get_type()` - zwraca typ czujnika
- `configure()` - konfiguruje czujnik za pomocą mapy parametrów
- `get_address()` - zwraca adres I2C czujnika

### Typy czujników i danych

Aby uprościć zarządzanie różnymi rodzajami czujników i typami danych, które oferują, zdefiniowałem dwie enumeracje: `SensorType` oraz `SensorDataTypeIdentifier`. Enumeracja `SensorType` określa rodzaj czujnika, natomiast `SensorDataTypeIdentifier` definiuje rodzaje danych, które mogą być odczytane z czujników.

```
enum class SensorType : uint8_t {
    NONE = 0x00,
    LIGHT = 0x01,
    ENVIRONMENT = 0x02,
};

enum class SensorDataTypeIdentifier : uint8_t {
```

```
    NONE = 0x00,
    LIGHT_LEVEL = 0x01,
    TEMPERATURE = 0x02,
    HUMIDITY = 0x03,
    PRESSURE = 0x04,
};
```

Listing 4.13: Definicje typów czujników i danych

Takie podejście pozwala na jednoznaczne identyfikowanie czujników i danych, co jest szczególnie istotne w systemie, który może zawierać wiele różnych czujników dostarczających różne rodzaje pomiarów.

#### Klasa SensorWrapper

Klasa `SensorWrapper` pełni rolę centralnego menedżera dla wszystkich czujników w systemie. Implementuje ona wzorzec projektowy Singleton, co zapewnia jednolity punkt dostępu do wszystkich czujników w całym systemie. Jest to szczególnie przydatne, gdy różne podsystemy satelity potrzebują dostępu do tych samych danych z czujników.

Klasa `SensorWrapper` udostępnia metody do inicjalizacji, konfiguracji i odczytywania danych z czujników.

### 4.8.3 Implementacja czujnika środowiskowego BME280

#### Klasa BME280

Klasa `BME280` implementuje niskopoziomową komunikację z czujnikiem poprzez interfejs I2C. Jest odpowiedzialna za inicjalizację sprzętu, konfigurację rejestrów, odczytywanie surowych danych pomiarowych oraz ich konwersję na wartości fizyczne w odpowiednich jednostkach.

Procedura inicjalizacji czujnika BME280 jest wieloetapowa i obejmuje:

1. Sprawdzenie poprawności inicjalizacji portu I2C
2. Weryfikację identyfikatora urządzenia (chip ID)

3. Konfigurację parametrów pracy czujnika
4. Odczytanie parametrów kalibracyjnych z czujnika

#### Konwersja danych pomiarowych

Jednym z najważniejszych aspektów działania czujnika BME280 jest konwersja surowych wartości odczytanych z rejestrów na rzeczywiste wartości fizyczne. W tym celu BME280 wykorzystuje zaawansowane algorytmy kompensacji, które uwzględniają parametry kalibracyjne zapisane w pamięci czujnika. W celu implementacji tych algorytmów, skorzystałem z oficjalnej dokumentacji producenta [9] oraz udostępnionych przez niego przykładowych implementacji.

#### Klasa opakowująca BME280Wrapper

Aby zintegrować czujnik BME280 z abstrakcyjnym interfejsem `ISensor`, stworzyłem klasę opakowującą `BME280Wrapper`. Klasa ta implementuje interfejs `ISensor` i przekierowuje wywołania metod do odpowiednich funkcji właściwego czujnika.

Warto zwrócić uwagę na metodę `read_data()`, która odczytuje wszystkie dane surowe jednocześnie (co jest bardziej efektywne), a następnie dokonuje odpowiedniej konwersji w zależności od żądanego typu danych. Jest to optymalizacja, która pozwala na zmniejszenie liczby operacji I2C, co przekłada się na większą dostępność czasową szyny dla innych czujników.

### 4.8.4 Implementacja czujnika światła BH1750

#### Klasa BH1750

Klasa `BH1750` implementuje niskopoziomową komunikację z czujnikiem poprzez interfejs I2C. Podobnie jak w przypadku BME280, jest odpowiedzialna za inicjalizację sprzętu, konfigurację rejestrów i odczytywanie danych pomiarowych.

Czujnik BH1750 oferuje różne tryby pracy, które różnią się rozdzielczością i częstotliwością pomiarów. Zaimplementowałem obsługę wszystkich trybów pracy poprzez enumerację `Mode` i metodę `configure()`.

#### Odczyt danych z czujnika BH1750

Odczyt danych z czujnika BH1750 jest znacznie prostszy niż w przypadku BME280, ponieważ czujnik ten dokonuje konwersji wartości surowej na luksy wewnętrznie. Wystarczy więc odczytać 2 bajty danych z rejestru i zastosować prosty wzór konwersji.

#### Klasa opakowująca BH1750Wrapper

Podobnie jak w przypadku BME280, stworzyłem klasę opakowującą `BH1750Wrapper`, która implementuje interfejs `ISensor` i przekierowuje wywołania metod do właściwego czujnika.

Interesującym aspektem implementacji klasy `BH1750Wrapper` jest metoda `configure()`, która pozwala na dynamiczną zmianę trybu pracy czujnika za pomocą mapy parametrów. Domyślnie czujnik jest konfigurowany w trybie ciągłym wysokiej rozdzielczości.

### 4.8.5 Implementacja klasy SensorWrapper

Implementacja klasy `SensorWrapper` obejmuje szereg metod, które umożliwiają zarządzanie czujnikami w systemie. Jedną z najważniejszych funkcjonalności jest inicjalizacja czujników, która jest realizowana przez metodę `sensor_init()`.

```
bool SensorWrapper::sensor_init(SensorType type, i2c_inst_t* i2c) {
    switch (type) {
    case SensorType::LIGHT:
        sensors[type] = new BH1750Wrapper(i2c);
        break;
    case SensorType::ENVIRONMENT:
        sensors[type] = new BME280Wrapper(i2c);
        break;
    default:
        return false;
    }
    return sensors[type]->init();
}
```

Listing 4.14: Inicjalizacja czujników w klasie SensorWrapper

Metoda ta tworzy odpowiednią instancję klasy opakowującej w zależności od typu czujnika, a następnie wywołuje metodę `init()`, która inicjalizuje sprzęt. Następnie czujnik jest przechowywany w mapie `sensors`, która mapuje typy czujników na ich instancje.

Kolejną ważną funkcjonalnością jest odczytywanie danych z czujników, które jest realizowane przez metodę `sensor_read_data()`:

```
float SensorWrapper::sensor_read_data(SensorType sensorType,
    SensorDataTypeIdentifier dataType) {
    if (sensors.find(sensorType) == sensors.end()) {
        return -1.0f;
    }
    return sensors[sensorType]->read_data(dataType);
}
```

Listing 4.15: Odczytywanie danych z czujników w klasie SensorWrapper

### 4.8.6 Integracja z systemem telemetrii

System czujników jest ściśle zintegrowany z systemem telemetrii satelity. Dane z czujników są regularnie odczytywane i zapisywane w rekordach telemetrycznych, które są następnie przechowywane w buforze i zapisywane na karcie SD. Szczegółowo opisałem tę funkcjonalność w rozdziale 4.6.3.

## 4.9 Zegar

Klasa `DS3231` stanowi interfejs do odczytu i konfiguracji układu DS3231. Jej głównym zadaniem jest zapewnienie abstrakcji sprzętowej, umożliwiając łatwy odczyt danych czasowych oraz konfigurację parametrów pracy układu.

W ramach tej klasy zaimplementowałem szereg funkcji, które obejmują odczyt

i zapis czasu i daty, odczyt temperatury, konwersję między formatami BCD i binarnym oraz obsługę stref czasowych . Konstruktor klasy `DS3231` przyjmuje jako argument wskaźnik na instancję `i2c_inst_t`, co daje dużą elastyczność w wyborze portu I2C, do którego podłączony jest układ DS3231.

Dodatkowo, w konstruktorze inicjalizowany jest mutex `clock_mutex_`, który pełni kluczową rolę w zapewnieniu bezpiecznego dostępu do zasobów klasy z różnych rdzeni procesora.

### 4.9.1 Odczyt i zapis czasu

Do odczytu czasu z układu DS3231 służy metoda `get_time()`, która pobiera dane z odpowiednich rejestrów układu i zapisuje je w strukturze `ds3231_data_t`. Struktura ta zawiera pola reprezentujące sekundy, minuty, godziny, dzień tygodnia, dzień miesiąca, miesiąc, rok oraz informację o stuleciu. Metoda `set_time()` pozwala na ustawienie czasu w układzie DS3231. Obie metody korzystają z funkcji `i2c_read_reg()` i `i2c_write_reg()` do komunikacji z układem poprzez interfejs I2C.

Układ DS3231 przechowuje czas w formacie BCD (Binary Coded Decimal), dlatego konieczna jest konwersja między formatami binarnym i BCD.

```
/**
 * @brief Gets current RTC time as Unix timestamp
 * @return Unix timestamp or -1 on error
 */
time_t DS3231::get_time() {
    uint8_t time_data[7];
    if (i2c_read_reg(DS3231_SECONDS_REG, 7, time_data) != 0) {
        uart_print("Failed to read time from RTC", VerbosityLevel::ERROR);
        return -1;
    }

    struct tm timeinfo = {};
    // Konwersja z formatu BCD na binarny i zapis do struktury tm
    timeinfo.tm_sec = ((time_data[0] >> 4) * 10) + (time_data[0] & 0x0F);
    timeinfo.tm_min = ((time_data[1] >> 4) * 10) + (time_data[1] & 0x0F);
```

```
    timeinfo.tm_hour = ((time_data[2] >> 4) * 10) + (time_data[2] & 0x0F);
    timeinfo.tm_mday = ((time_data[4] >> 4) * 10) + (time_data[4] & 0x0F);
    timeinfo.tm_mon = (((time_data[5] & 0x1F) >> 4) * 10) + (time_data[5] & 0x0F
    ) - 1;
    timeinfo.tm_year = ((time_data[6] >> 4) * 10) + (time_data[6] & 0x0F) + 100;
    timeinfo.tm_isdst = 0;

    time_t unix_time = mktime(&timeinfo);
    if (unix_time == -1) {
        uart_print("Failed to convert RTC time", VerbosityLevel::ERROR);
        return -1;
    }

    return unix_time;
}
```

Listing 4.16: Funkcja pobierająca czas z układu DS3231

### 4.9.2 Strefa czasowa

Klasa `DS3231` umożliwia ustawienie strefy czasowej za pomocą metody `set_timezone_offset()`. Przesunięcie strefy czasowej jest przechowywane w zmiennej `timezone_offset_minutes_` i jest uwzględniane podczas odczytu czasu lokalnego za pomocą metody `get_local_time()`.

### 4.9.3 Temperatura

Pomiar temperatury jest używany przez układ głównie do kompensacji temperaturowej oscylatora kwarcowego, jednak producent daje możliwość odczytu tej wartości przez interfejs I2C. Klasa `DS3231` udostępnia metodę `get_temperature()`, która zwraca temperaturę w stopniach Celsjusza z dokładnością do 0.25 °C. Dzięki temu można monitorować temperaturę wewnątrz obudowy satelity.

### 4.9.4 Polecenia zegara

| ID | Format komendy | Opis |
|---|---|---|
| 3.0 | KBST;0;GET;3;0;;TSBK | Pobiera aktualny czas. Zwraca czas w formacie Unix timestamp. |
| | KBST;0;SET;3;0;TIMESTAMP;TSBK | Ustawia aktualny czas. Oczekuje Unix timestamp jako parametr. |
| 3.1 | KBST;0;GET;3;1;;TSBK | Pobiera aktualne przesunięcie strefy czasowej w minutach. |
| | KBST;0;SET;3;1;OFFSET;TSBK | Ustawia przesunięcie strefy czasowej w minutach (od -720 do +720). |
| 3.4 | KBST;0;GET;3;4;;TSBK | Pobiera temperaturę wewnętrzną z czujnika DS3231. Zwraca temperaturę w stopniach Celsjusza. |

Tabela 4.9: Komendy systemu zegara

## 4.10 System lokalizacji i komunikacji GPS

### 4.10.1 Wprowadzenie

System lokalizacji stanowi kluczowy element funkcjonalny, który zaprojektowałem, aby umożliwić precyzyjne określanie pozycji satelity. W niniejszym rozdziale omówię szczegółowo architekturę systemu lokalizacji, mechanizmy przetwarzania danych GPS oraz implementację protokołu NMEA, który jest standardem komunikacji z modułami GPS.

Głównym celem systemu lokalizacji jest ciągły odbiór, przetwarzanie i udostępnianie danych pozycyjnych z modułu GPS dla innych podsystemów satelity.

### 4.10.2 Architektura systemu lokalizacji

System lokalizacji składa się z kilku kluczowych komponentów:

- Moduł odbiornika GPS podłączony do mikrokontrolera poprzez interfejs UART
- Kolektor danych GPS (`gps_collector`) odpowiedzialny za odczytywanie surowych danych z UART
- Klasa `NMEAData` przechowująca przetworzone dane z protokołu NMEA
- Zestaw komend do zarządzania i odczytu danych GPS

Najważniejszym elementem systemu jest moduł odbiorczy GPS, który komunikuje się z mikrokontrolerem poprzez dedykowany port UART. Komunikacja odbywa się przy wykorzystaniu standardowego protokołu NMEA 0183 [1], który definiuje format komunikatów wysyłanych przez odbiornik GPS. Zdecydowałem się na wykorzystanie tego protokołu ze względu na jego powszechne zastosowanie w przemyśle oraz dostępność gotowych bibliotek do jego obsługi.

### 4.10.3 Protokół NMEA i jego obsługa

Protokół NMEA 0183 jest standardowym protokołem komunikacyjnym używanym przez większość odbiorników GPS. Protokół ten definiuje format komunikatów tekstowych, które składają się z sekwencji ASCII rozpoczynających się znakiem $ i zakończonych znakami końca linii (\r\n). Każda wiadomość protokołu NMEA zawiera określony typ danych identyfikowany przez prefiks po znaku $.

#### Typy komunikatów NMEA

W systemie lokalizacji satelity KubiSat zdecydowałem się na przetwarzanie dwóch najważniejszych typów komunikatów NMEA:

- **GPRMC** (Recommended Minimum Navigation Information) - zawiera podstawowe informacje nawigacyjne takie jak czas, data, pozycja, prędkość i kurs
- **GPGGA** (Global Positioning System Fix Data) - zawiera dane o jakości fixa GPS, liczbie widocznych satelitów i wysokości nad poziomem morza

Poniżej przedstawiam przykład komunikatu GPRMC:

```
$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A
```

Gdzie:

- `123519` - czas UTC w formacie hhmmss (12:35:19)
- `A` - status (`A` = aktywny, `V` = nieważny)
- `4807.038,N` - szerokość geograficzna (48° 07.038' N)
- `01131.000,E` - długość geograficzna (11° 31.000' E)
- `022.4` - prędkość w węzłach
- `084.4` - kierunek w stopniach
- `230394` - data w formacie ddmmyy (23.03.1994)
- `003.1,W` - deklinacja magnetyczna
- `*6A` - suma kontrolna

#### Klasa NMEAData

Do przechowywania i zarządzania danymi z protokołu NMEA stworzyłem klasę `NMEAData`, która implementuje wzorzec projektowy Singleton. Klasa ta przechowuje tokeny z ostatnio odebranych komunikatów GPRMC i GPGGA oraz udostępnia metody do ich aktualizacji i odczytu. Dzięki zastosowaniu wzorca Singleton, dane GPS są dostępne dla wszystkich podsystemów satelity za pomocą jednej instancji klasy.

Zdecydowałem się na zabezpieczenie dostępu do danych GPS za pomocą muteksów, ponieważ dane te mogą być aktualizowane i odczytywane z różnych wątków. Muteksy zapewniają, że operacje odczytu i zapisu nie będą się nakładać, co mogłoby prowadzić do niespójności danych.

### 4.10.4 Kolektor danych GPS

Głównym elementem systemu odpowiedzialnym za zbieranie danych z odbiornika GPS jest funkcja `collect_gps_data()`. Funkcja ta jest wywoływana cyklicznie w głównej pętli programu i odpowiada za:

- Odczytywanie surowych danych z portu UART podłączonego do odbiornika GPS
- Buforowanie danych i wykrywanie kompletnych komunikatów NMEA
- Przetwarzanie komunikatów NMEA i aktualizację danych w klasie `NMEAData`

```
void collect_gps_data() {

if (SystemStateManager::get_instance().is_gps_collection_paused()) {
    return;
}

std::array<char, MAX_RAW_DATA_LENGTH> raw_data_buffer;
static int raw_data_index = 0;

while (uart_is_readable(GPS_UART_PORT)) {
    char c = uart_getc(GPS_UART_PORT);

    if (c == '\r' || c == '\n') {
        // End of message
        if (raw_data_index > 0) {
            raw_data_buffer[raw_data_index] = '\0';
            std::string message(raw_data_buffer.data());
            raw_data_index = 0;

            // Split the message into tokens
            std::vector<std::string> tokens = splitString(message, ',');

            // Update the global vectors based on the sentence type
            if (message.find("$GPRMC") == 0) {
                NMEAData::get_instance().update_rmc_tokens(tokens);
            } else if (message.find("$GPGGA") == 0) {
                NMEAData::get_instance().update_gga_tokens(tokens);
            }
        }
    } else {
```

```
            // Append to buffer
            if (raw_data_index < MAX_RAW_DATA_LENGTH - 1) {
                raw_data_buffer[raw_data_index++] = c;
            } else {
                raw_data_index = 0;
            }
        }
    }
}
```

Listing 4.17: Funkcja zbierająca dane GPS

Stan systemowy `is_gps_collection_paused` jest sprawdzany na początku funkcji, aby w przypadku jego aktywacji zatrzymać zbieranie danych GPS - jest to konieczne w sytuacji aktywnego trybu przekierowania UART (opisanego w 4.10.5).

### 4.10.5 Komendy systemu lokalizacji

System lokalizacji udostępnia następujące komendy, które mogą być wykorzystane do zarządzania modułem GPS:

#### Sterowanie zasilaniem modułu GPS

Funkcja `handle_gps_power_status()` obsługuje komendę sterowania zasilaniem modułu GPS. Pozwala ona na włączenie lub wyłączenie modułu GPS, co jest istotne dla oszczędzania energii w okresach, gdy dane pozycyjne nie są potrzebne.

To polecenie jest dostępne tylko w trybie naziemnym, opisanym w 4.3.3 - w trybie lotu nie ma możliwości ręcznego wyłączenia zasilania.

Algorytm funkcji `handle_gps_power_status()` realizuje następującą logikę:

W pierwszym kroku funkcja inicjalizuje pustą tablicę ramek odpowiedzi. Następnie, zależnie od typu operacji, wykonuje odpowiednie działania.

Dla operacji typu SET (ustawianie stanu zasilania):

1. Sprawdza, czy system pracuje w trybie zasilania bateryjnego. Jeśli tak, zwraca

| ID | Format komendy | Opis |
|---|---|---|
| 7.1 | KBST;0;GET;7;1;;TSBK | Pobiera aktualny stan zasilania modułu GPS (włączony/wyłączony). |
| 7.1 | KBST;0;SET;7;1;POWER;TSBK | Ustawia stan zasilania modułu GPS. POWER=0 (wyłączony), POWER=1 (włączony). |
| 7.2 | KBST;0;SET;7;2;TIMEOUT;TSBK | Włącza tryb transparentny UART dla modułu GPS. TIMEOUT określa czas trwania trybu w sekundach (domyślnie 60s). W trybie transparentnym można bezpośrednio komunikować się z modułem GPS. Aby wyjść z trybu przed upływem limitu czasu, należy wysłać sekwencję "##EXIT##". |

Tabela 4.10: Komendy systemu lokalizacji

błąd INVALID_OPERATION, ponieważ zmiana stanu zasilania GPS jest niedozwolona w tym trybie.

2. Weryfikuje, czy parametr jest pusty. W przypadku braku parametru zwraca błąd PARAM_REQUIRED, ponieważ wartość stanu zasilania jest wymagana.

3. Próbuje skonwertować parametr na wartość liczbową całkowitą. Jeśli konwersja nie powiedzie się lub wartość nie jest równa 0 lub 1, zwraca błąd PARAM_INVALID.

4. Po pomyślnej walidacji ustawia pin zasilający moduł GPS na żądaną wartość (0 - wyłączony, 1 - włączony).

5. Emituje odpowiednie zdarzenie systemowe (POWER_ON lub POWER_OFF).

6. Zwraca odpowiedź sukcesu wraz z informacją o nowym stanie zasilania.

Dla operacji typu GET (odczyt stanu zasilania):

1. Sprawdza, czy parametr nie jest pusty. Jeśli zawiera jakąś wartość, zwraca błąd PARAM_UNNECESSARY, ponieważ operacja odczytu nie wymaga parametrów.

2. Odczytuje aktualny stan pinu zasilającego moduł GPS.

3. Zwraca odpowiedź zawierającą aktualny stan zasilania modułu GPS.

Taka implementacja zapewnia bezpieczną kontrolę nad zasilaniem modułu GPS, uwzględniając aktualny tryb pracy systemu oraz poprawność parametrów wejściowych.

#### Tryb transparentny UART

Funkcja `handle_enable_gps_uart_passthrough()` umożliwia wejście w tryb transparentny, w którym komunikacja z modułem GPS odbywa się bezpośrednio przez port UART. Jest to szczególnie przydatne podczas debugowania i konfiguracji modułu GPS, ponieważ pozwala na wysyłanie specjalnych komend konfiguracyjnych do odbiornika przez oprogramowanie producenta.

Ten tryb jest dostępny tylko w trybie naziemnym 4.3.3, ponieważ jego aktywacja blokuje zbieranie danych GPS przez system lokalizacji na czas trwania trybu.

Funkcja `handle_enable_gps_uart_passthrough()` realizuje następujący algorytm:

1. Sprawdzenie typu operacji (musi być SET).
2. Wyłączenie komendy w trybie bateryjnym.
3. Parsowanie i walidacja parametru timeout.
4. Konfiguracja parametrów UART i sekwencji wyjścia.
5. Wstrzymanie zbierania danych GPS.
6. Emisja zdarzenia rozpoczęcia trybu.
7. Zmiana baudrate UART na baudrate GPS.
8. Przekazywanie danych między portami UART.
9. Przywrócenie oryginalnego baudrate UART.
10. Wznowienie zbierania danych GPS.
11. Emisja zdarzenia zakończenia trybu.
12. Budowa i zwrot ramki odpowiedzi.

# System operatora satelity

W celu ułatwienia zdalnej komunikacji z modelem stworzyłem prosty system służący do wysyłania poleceń i odbierania danych z satelity. System ten składa się z płytki Raspberry Pi RP2040-Zero, modułu LoRa SX1278, identycznego jak w satelicie, oraz z konwertera USB-UART służącego do komunikacji z komputerem. Funkcję konwertera realizuje prosty i powszechny układ CH340N, który emuluje port szeregowy na komputerze.

Prosty program pracujący w pamięci mikrokontrolera RP2040 odpowiada za dwustronną komunikację z modułem LoRa. Wysyła on dane odebrane z komputera przez port szeregowy do modułu LoRa i odbiera dane z modułu LoRa, które przekazuje do komputera. Główna pętla programu jest wywoływana praktycznie od razu po podłączeniu konwertera do komputera, co pozwala na natychmiastową komunikację z modelem.

Istnieje możliwość wysyłania ramek komunikacji bezpośrednio na utworzony wirtualny port COM, jednak w celu ułatwienia interakcji z użytkownikiem stworzyłem prosty interfejs graficzny bazujący na języku C# i frameworku WPF (Windows Presentation Foundation). Interfejs ten pozwala na wysyłanie komend do satelity, odbieranie odpowiedzi, wizualizację danych telemetrycznych, zarządanie konfiguracją systemu oraz podgląd komunikacji w czasie rzeczywistym.

Ponadto program pozwala na eksport wizualizacji telemetrii do formatu PNG, zapisywanie logów komunikacji do pliku tekstowego a co najważniejsze, umożliwia wczytanie danych telemetrycznych z pliku i ich wizualizację. Dzięki temu użytkownik może analizować dane telemetryczne zebrane na kartę SD w trakcie lotu po jego zakończeniu, bez konieczności korzystania z łączności radiowej z modelem.

## 5.1 Warstwa sprzętowa

## 5.2 Warstwa programowa

### 5.2.1 Wprowadzenie

Program został zaprojektowany z myślą o zapewnieniu prostego w obsłudze interfejsu do monitorowania i kontrolowania satelity poprzez port szeregowy, realizując komunikację w oparciu o protokół LoRa.

### 5.2.2 Architektura systemu

#### Model komunikacji

W sercu systemu znajduje się model komunikacji oparty na protokole tekstowym, który zaimplementowałem przy pomocy klas `SerialCommunicator` i `CommandManager`.

#### Struktura klas

System `kubisat_operator` zorganizowałem wokół następujących kluczowych klas:

- `SerialCommunicator` - odpowiada za niskopoziomową komunikację przez port szeregowy, buforowanie danych oraz wykrywanie kompletnych ramek.
- `CommandManager` - implementuje mechanizm asynchronicznego wywołania komend i oczekiwania na odpowiedzi przy użyciu `TaskCompletionSource`.
- `CommandDefinition` - definiuje strukturę komendy wraz z jej identyfikatorami, nazwą, uprawnieniami dostępu i opisem.
- `CommandRegistry` - centralny rejestr wszystkich dostępnych komend w systemie.
- `CommandViewModel` - implementuje wzorzec MVVM dla komend, umożliwiając powiązanie logiki aplikacji z interfejsem użytkownika.

Wykorzystałem architekturę MVVM (Model-View-ViewModel), która umożliwia separację logiki biznesowej od warstwy prezentacji, co znacząco ułatwia testowanie i rozbudowę systemu.

### 5.2.3 Asynchroniczny system obsługi komend

#### Mechanizm wywołania i oczekiwania

Jednym z kluczowych elementów, który zaimplementowałem, jest asynchroniczny system obsługi komend. Zastosowałem nowoczesne podejście oparte na `async/await` w C#, które eliminuje złożoną logikę obsługi wywołań zwrotnych (callbacks).

Każde wywołanie komendy zwraca `Task<string>`, który jest realizowany dopiero po otrzymaniu odpowiedzi z satelity. Przykładowo:

```
public async Task<string> ExecuteCommandAsync(CommandDefinition command,
                                               string parameter = "",
                                               string operationType = "GET")
{
    // Utworzenie klucza komendy
    string commandKey = $"{command.GroupId}-{command.Id}-{operationType}";

    // Utworzenie źródła zakończenia zadania (TaskCompletionSource)
    var commandTask = new TaskCompletionSource<string>();
    _pendingCommands[commandKey] = commandTask;

    // Wysłanie komendy
    await _serialCommunicator.SendFrameAsync(CreateCommandFrame(...));

    // Oczekiwanie na odpowiedź z timeout'em
    return await commandTask.Task.WaitAsync(TimeSpan.FromSeconds(10));
}
```

Listing 5.1: Asynchroniczne wywołanie komendy

Gdy odpowiedź z satelity zostanie odebrana, system automatycznie identyfikuje

odpowiednią `TaskCompletionSource` i sygnalizuje jej zakończenie, co wznawia wykonanie kodu czekającego na wynik.

#### Obsługa błędów i timeout'ów

W ramach systemu zaimplementowałem kompleksową obsługę błędów, która uwzględnia:

- Timeout komunikacji (maksymalnie 2 sekundy oczekiwania na odpowiedź)
- Błędy raportowane przez satelitę (ramki typu ERR)
- Wyjątki związane z komunikacją szeregową
- Błędy parsowania danych

Dzięki mechanizmowi wyjątków C# i konstrukcji `try/catch`, każdy błąd jest odpowiednio przechwytywany i prezentowany użytkownikowi w sposób przystępny.

### 5.2.4 Wizualizacja danych

Żądanie pobrania ostatniego rekordu telemtrii lub odczytów środowiskowych z satelity automatycznie wypełnia pola tabeli w interfejsie użytkownika. Zaimplementowałem mechanizm automatycznego odświeżania danych co 1 sekundę, co pozwala na monitorowanie zmian w czasie rzeczywistym - w tym przypadku dane są na bieżąco wizualizowane na wykresach.

#### System wykresów

Dla efektywnego monitorowania parametrów satelity, zaimplementowałem system wizualizacji danych oparty na bibliotece OxyPlot [4]. Dane są organizowane w kilku kategoriach:

- Telemetria zasilania (napięcia, prądy ładowania i rozładowania)
- Dane sensoryczne (temperatura, ciśnienie, wilgotność, natężenie światła)

- Dane pozycyjne GPS

Dla każdego typu danych utworzyłem dedykowane klasy zarządzające wykresami, takie jak `TelemetryPlotManager` i `SensorPlotManager`, które obsługują:

- Dynamiczne dodawanie punktów danych
- Automatyczne skalowanie osi
- Obsługę legendy
- Eksport wykresów do plików PNG

**Wizualizacja lokalizacji**

W przypadku pobrania jednego rekordu z lokalizacją GPS, system automatycznie wyświetla link przekierowujący do mapy Google z zaznaczoną pozycją, natomiast po zakończeniu trybu monitorowania, system generuje plik KML, który można zaimportować do Google Earth w celu wizualizacji trasy w trzech wymiarach. Plik KML można także wygenerować dla zaimportowanych danych historycznych.

### 5.2.5 Zarządzanie danymi

System obsługuje następujące formaty danych:

- Format CSV dla importu/eksportu danych telemetrycznych i sensorycznych
- Format tekstowy dla logów komunikacji
- Format PNG dla eksportu wykresów

Dane są przechowywane w strukturach takich jak `TelemetryRecord` i `SensorDataRecord`, które enkapsulują wszystkie istotne parametry wraz z metodami konwersji.

System automatycznie generuje nazwy plików zawierające datę i czas, ułatwiając organizację danych.

### 5.2.6 Interfejs użytkownika

#### Organizacja interfejsu

Interfejs użytkownika zorganizowałem wokół zakładek tematycznych, odpowiadających głównym funkcjom systemu:

- Parametry systemowe - ustawienia i diagnostyka
- GPS - kontrola modułu GPS i wizualizacja pozycji
- Telemetria - monitoring parametrów zasilania i pracy systemu
- Sensory - dane z czujników środowiskowych
- Log komunikacji - monitorowanie surowej komunikacji

Każda zakładka zawiera odpowiednie elementy sterujące i prezentacyjne, zorganizowane w logiczne grupy.

#### Kolorowanie logów

Dla ułatwienia analizy komunikacji, zaimplementowałem system kolorowania logów w monitorze komunikacji:

- Kolor niebieski - komunikaty przychodzące (kierunek = 1)
- Kolor zielony - komunikaty wychodzące (kierunek = 0)
- Kolor czarny - pozostałe informacje diagnostyczne

System obsługuje również opcjonalne znaczniki czasu oraz automatyczne przewijanie do najnowszych wpisów.

### 5.2.7 Automatyczne wykrywanie urządzeń

Zaimplementowałem mechanizm automatycznego wykrywania zmian w dostępnych portach szeregowych, wykorzystując klasę `ManagementEventWatcher` z przestrzeni nazw `System.Management`. System reaguje na podłączenie lub odłączenie urządzeń, dynamicznie aktualizując listę dostępnych portów.

### 5.2.8 Mechanizmy odświeżania danych

Dla zapewnienia aktualności informacji, zaimplementowałem kilka mechanizmów odświeżania danych:

- Ręczne odświeżanie po kliknięciu odpowiednich przycisków
- Automatyczne odświeżanie z konfigurowalnymi czasem trwania dla telemetrii i danych sensorycznych
- Pasek postępu wizualizujący pozostały czas

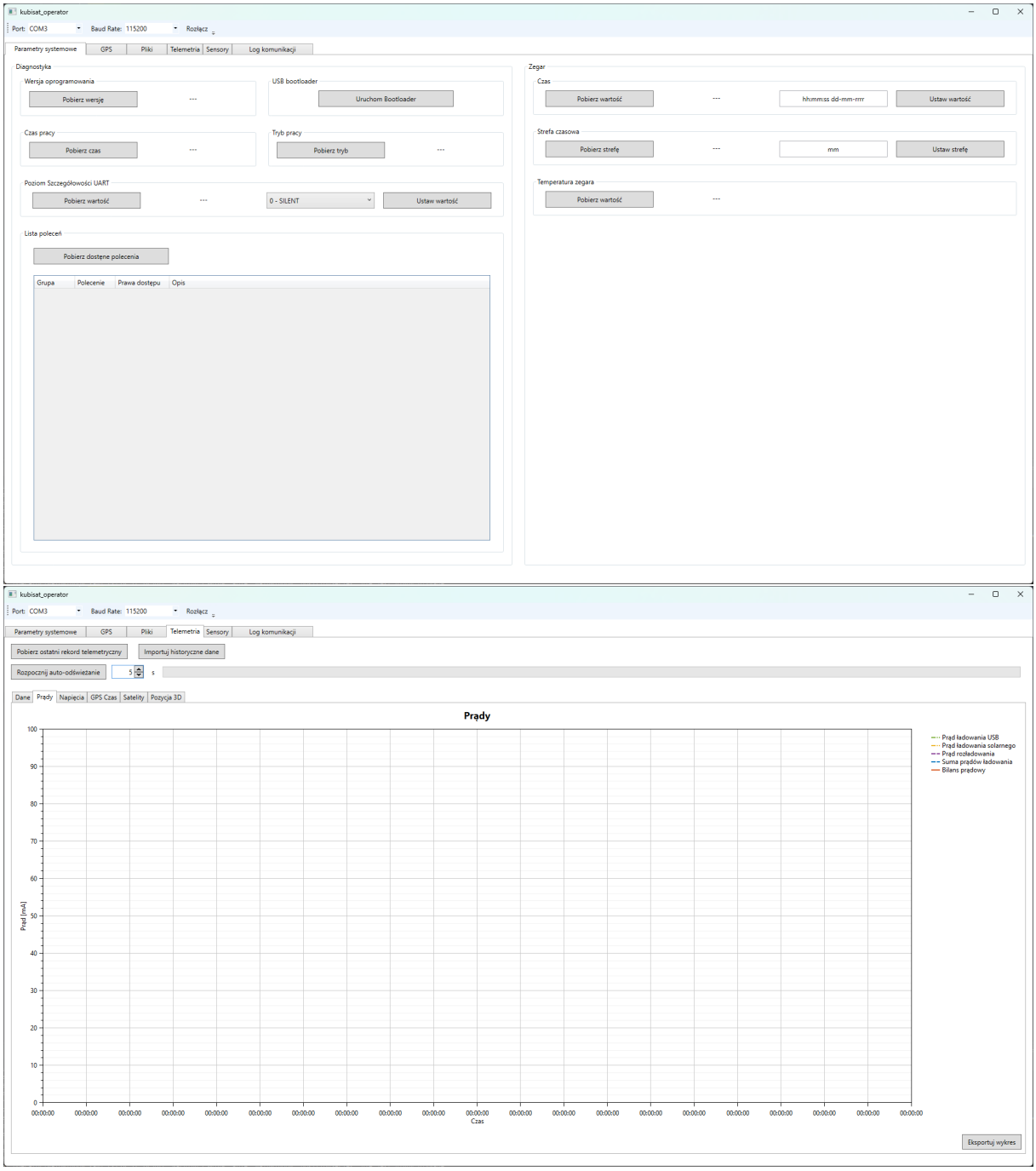
Mechanizm ten opiera się na klasie `DispatcherTimer`, która zapewnia wykonywanie operacji odświeżania w kontekście wątku UI. Po ustawieniu czasu odświeżania, system automatycznie aktualizuje dane i wykresy co 1 sekundę poprzez wysłanie zapytania do satelity o aktualne wartości.

### 5.2.9 Informacje o jakości połączenia

Na górze okna programu umieściłem przełącznik diagnostyki połączenia - po włączeniu tej opcji, do każdej ramki przesyłanej przez port szeregowy do stacji naziemnej dodawany jest wcześniej ustalony znak specjalny - w tym przypadku znak zapytania (?). Po otrzymaniu ramki z tym znakiem, program ustawia flagę żadania informacji o jakości połączenia, a następnie po oedbraniu ramki odpowiedzi z satelity, dołącza do niej informacje o RSSI i SNR, a następnie przekazuje do portu UART. Program na stacji naziemnej odczytuje te dane i prezentuje je w interfejsie użytkownika.

### 5.2.10 Zrzuty ekranu

W tej sekcji prezentuję kilka zrzutów ekranu z działania systemu `kubisat_operator`, które ilustrują główne funkcje i interfejs użytkownika.




Rysunek 23: Główne okno programu `kubisat_operator`

# Testy

## 6.1 Testy poleceń

Testy poprawności działania systemu komunikacji oraz zależności między systemami wykonałem używając do tego celu zakładki Testy w programie Operator 5.

Każde z dostępnych poleceń przetestowałem pod kątem poprawności działania, poprzez wysłanie odpowiednich komend do modelu i sprawdzenie czy odpowiedź jest zgodna z oczekiwaniami. Sprawdziłem czy polecenia mają prawa dostepu zgodne z założeniami, poprawność odpowiedzi, formatowanie oraz czas odpowiedzi systemu na polecenie.

## 6.2 Testy telemetrii

Testy systemu telemetrii przeprowadziłem w formie ograniczonej symulacji lotu modelu. W tym celu wykonałem przejazd samochodem z włączonym modelem na trasie o długości około 17km Satelita pracował cały czas na własnym zasilaniu, zgodnie z założeniami system telemetrii rejestrował dane z modułu GPS, pomiary parametrów zasilania oraz dane z czujników.

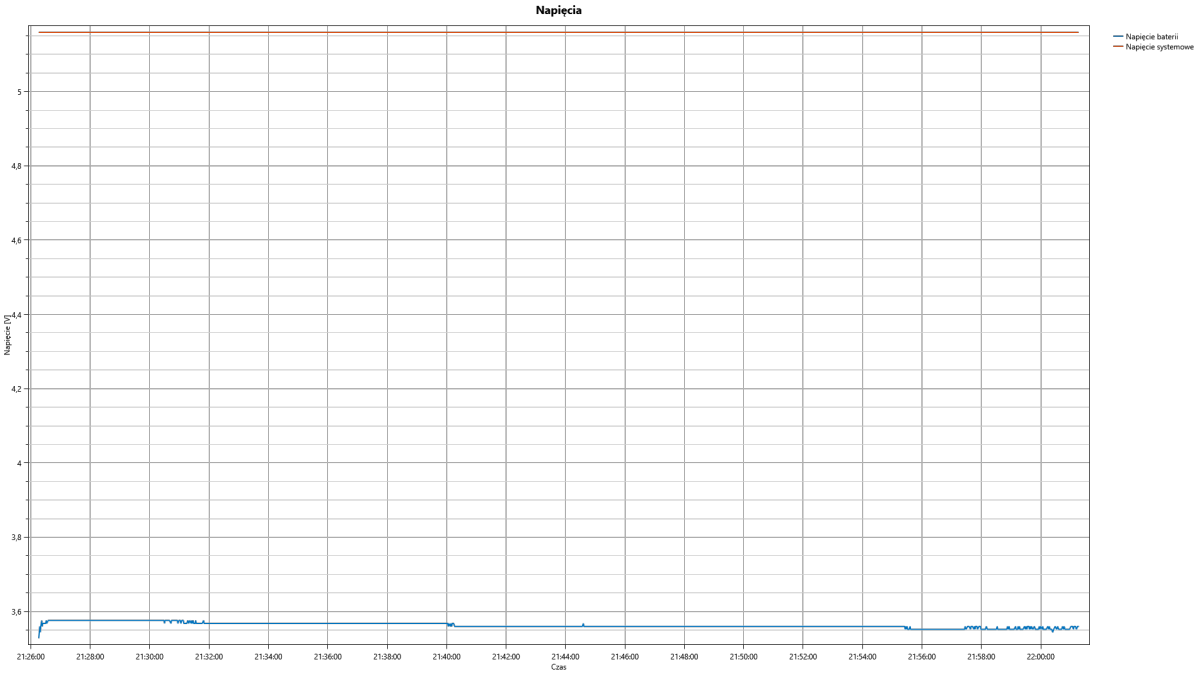
Korzystając z programu Operator (opis w 5) odczytałem zapisane dane z pamięci urządzenia. Dane zostały zapisane w pliku CSV, który następnie zaimportowałem jako dane historyczne do programu w celu analizy. Poniżej przedstawiam wyniki testów w formie wykresów, tabel oraz śladu GPS.

Na wykresie natężenia światła oraz temperatury i wilgotności około godziny 21:58 widać nagłe zmiany wartości. Wynika to z faktu że w tym momencie przenosiłem model z samochodu do domu, co spowodowało zmianę warunków środowiskowych.

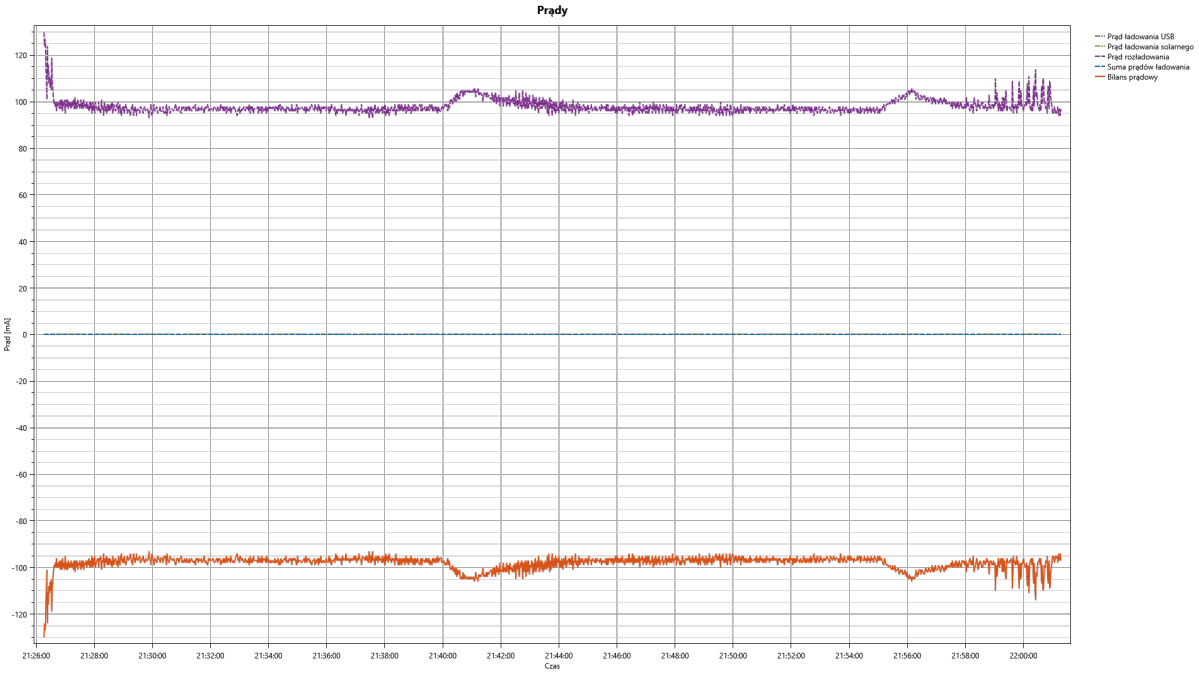
Skoki pobieranego prądu zasilania wynikają z pracy modułu radiowego - po przeniesieniu modelu do domu odczytałem zdalnie jego parametry w celu sprawdzenia poprawności działania.



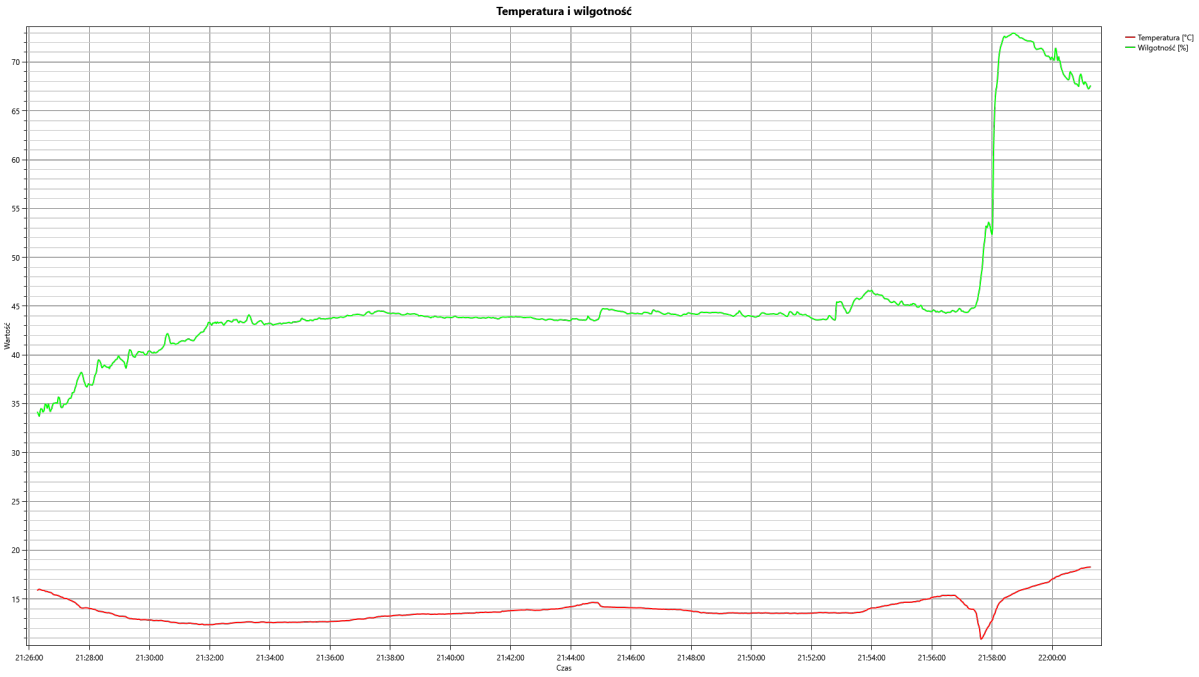

Rysunek 24: Ślad GPS



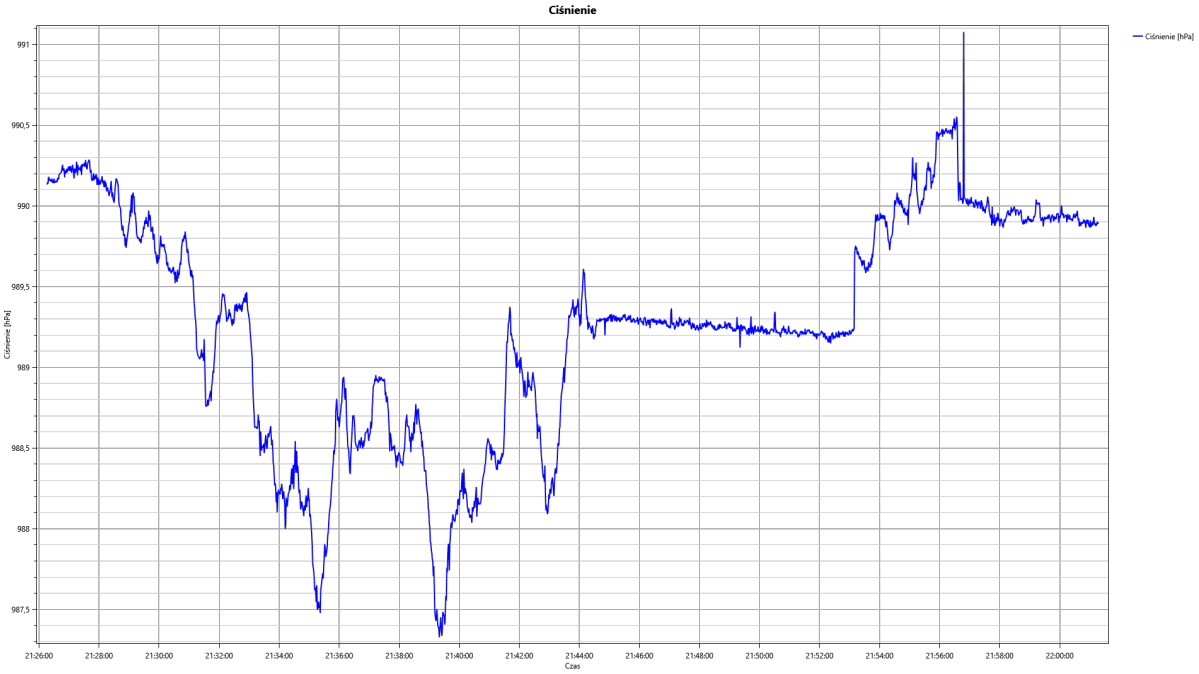

Rysunek 25: Pomiary napięcia baterii oraz zasilania



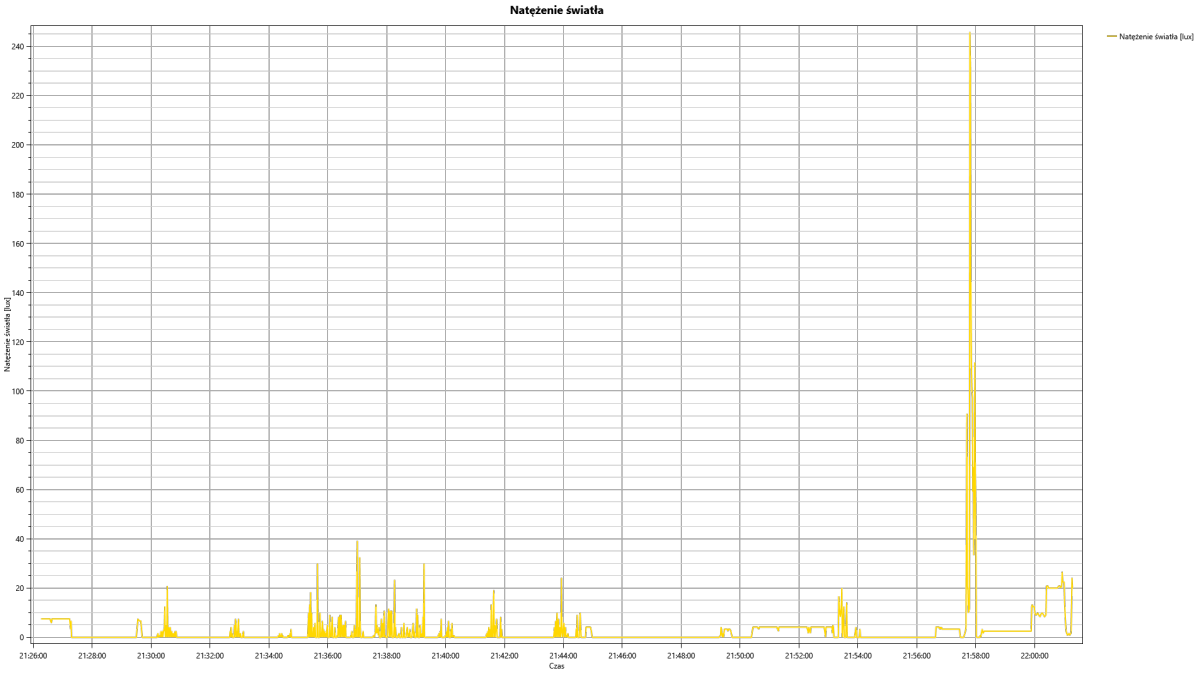

Rysunek 26: Pomiary prądów w systemie




Rysunek 27: Pomiary temperatury i wilgotności




Rysunek 28: Pomiary ciśnienia atmosferycznego




Rysunek 29: Pomiary poziomu naświetlenia

# Podsumowanie

## 7.1 Wnioski

Celem pracy było zaprojektowanie i zbudowanie systemu komunikacji z satelitą typu CubeSat. System miał umożliwić zdalne sterowanie satelitą, pobieranie danych telemetrii oraz konfigurację parametrów pracy. Postawione cele zrealizowałem a system działa zgodnie z założeniami co potwierdzają testy przeprowadzone na modelu satelity.

## 7.2 Możliwości rozwoju

System komunikacji z satelitą jest w pełni funkcjonalny, jednak istnieje wiele możliwości rozwoju i ulepszenia. Poniżej przedstawiam kilka propozycji rozwoju systemu:

- Rozbudowa systemu o dodatkowe moduły komunikacji, takie jak moduł GSM, który umożliwiłby komunikację z satelitą w dowolnym miejscu na ziemi.
- Implementacja systemu w oparciu o FreeRTOS, co pozwoliłoby na zwiększenie stabilności systemu oraz możliwość uruchomienia wielu zadań równolegle.
- Implementacja trybów oszczędzania energii, które pozwoliłyby na zwiększenie czasu pracy satelity.
- Rozbudowa systemu o dodatkowe czujniki, które pozwoliłyby na zbieranie dodatkowych danych telemetrii.
- Implementacja systemu kontroli dostępu, który pozwoliłby na zabezpieczenie dostępu do systemu przed niepowołanymi osobami.
- Rozbudowa systemu o prostą kamerę, która pozwoliłaby na wykonywanie zdjęć z orbity.

Dla docelowego systemu satelitarnego możliwe jest również rozszerzenie funkcjonalności o polecenia pobierania plików z karty SD, a także system automatycznej

synchronizacji czasu z GPS, automatycznego wykrywania i inicjalizacji nowych modułów w trakcie pracy systemu oraz rozbudowa systemu zarządzania energią w celu zwiększenia czasu pracy satelity.

Aby ładowanie z ogniw fotowołtaicznych było bardziej efektywne, należałoby zmienić ich konfigurację na większą ilość paneli mniejszego rozmiaru, co przez zwiększenie napięcia zwiększyłoby sprawność ładowania.

# Bibliografia


[1] National Marine Electronics Association. Nmea 0183 standard for interfacing marine electronic devices. https://www.plaisance-pratique.com/IMG/pdf/NMEA0183-2.pdf, 2002. Version 3.01, dostęp 03-2025.

[2] Akshaya Bali. Raspberry pi pico lora library. https://github.com/akshayabali/pico-lora, 2021. MIT, dostęp 12-2024.

[3] Shenzhen H&M Semiconductor Co.Ltd. Dw01a. https://hmsemi.com/downfile/dw01a.pdf, 2023. Dostęp 11-2024.

[4] OxyPlot Contributors. Oxyplot: Cross-platform plotting library for .net, 2024. Dostęp 03-2025.

[5] Doroshkin, Alexander, Zadorozhny, Alexander, Kus, Oleg, Prokopyev, Vitaliy, and Prokopyev, Yuri. Laboratory testing of lora modulation for cubesat radio communications. *MATEC Web Conf.*, 158:01008, 2018.

[6] Texas Instruments. Ina3221. https://www.ti.com/lit/ds/symlink/ina3221.pdf, 2012. SBOS576B, dostęp 11-2024.

[7] Hiroyuki OYAMA. pico-vfs: Virtual file system for raspberry pi pico. https://github.com/raspberrypi/pico-vfs, 2024. BSD-3-Clause, dostęp 02-2025.

[8] Semtech. Sx1278. https://semtech.my.salesforce.com/sfc/p/#E0000000JelG/a/2R0000001Rc1/QnUuV9TviODKUgt_rpBlPz.EZA_PNK7Rpi8HA5.Sbo, 2020. Rev. 7 - May 2020, dostęp 11-2024.

[9] Bosch Sensortec. Bme280. https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf, 2015. BST-BME280-DS002, dostęp 11-2024.

[10] Ltd. Shanghai Natlinear Electronics Co. Ln2220 2a high efficiency boost dc/dc voltage regulator. https://www.lcsc.com/datasheet/lcsc_datasheet_2105242005_NATLINEAR-LN2220PAR_C2833712.pdf, 2014. Rev. 1.0, dostęp 11-2024.

[11] The Things Network Global Team. Lorawan® distance world record broken, twice. 766 km (476 miles) using 25mw transmission power. https://www.

thethingsnetwork.org/article/lorawan-distance-world-record, 2019, dostęp 11-2024.

[12] u blox. Neo-6m. https://www.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_%28GPS.G6-HW-09005%29.pdf, 2015. GPS.G6-HW-09005, dostęp 12-2024.

[13] California Polytechnic State University. Cubesat design specification rev. 13. https://static1.squarespace.com/static/5418c831e4b0fa4ecac1bacd/t/56e9b62337013b6c063a655a/1458157095454/cds_rev13_final2.pdf, 2014. The CubeSat Program, Cal Poly SLO, dostęp 09-2024.

# Spis rysunków


1 Schemat blokowy . . . . . 8

2 Diagram modułu zasilania . . . . . 10

3 Schemat układu pomiarowego . . . . . 12

4 Schemat zabezpieczenia akumulatorów . . . . . 13

5 Schemat układu ładowania USB . . . . . 14

6 Schemat układu ładowania paneli fotowoltaicznych . . . . . 15

7 Charakterystyka przetwornicy LN2220 . . . . . 16

8 Schemat przetwornicy . . . . . 16

9 Schemat połączeń procesora . . . . . 18

10 Schemat połączeń modułu radiowego . . . . . 19

11 Schemat połączeń modułu GPS . . . . . 21

12 Schemat połączeń zegara czasu rzeczywistego . . . . . 22

13 Schemat połączeń karty pamięci SD . . . . . 23

14 Schemat połączeń modułu czujników . . . . . 24

15 Wymiary podstawy . . . . . 27

16 Wymiary elementu narożnego . . . . . 27

17 Obudowa podzielona na dwie cześci . . . . . 28

18 Obudowa zmontowana . . . . . 28

19 Częściowo otwarta obudowa . . . . . 29

20 Obudowa kompletna bez ogniw . . . . . 29

21 Obudowa z ogniwami fotowoltaicznymi . . . . . 30

22 Schemat komunikacji w systemie . . . . . 35

23 Główne okno programu `kubisat_operator` . . . . . 79

24 Ślad GPS . . . . . 81

25 Pomiary napięcia baterii oraz zasilania . . . . . 81
26 Pomiary prądów w systemie . . . . . 82
27 Pomiary temperatury i wilgotności . . . . . 82
28 Pomiary ciśnienia atmosferycznego . . . . . 83
29 Pomiary poziomu naświetlenia . . . . . 83

# Spis listingów

4.1 Stany systemowe . . . . . 39
4.2 Klasa PowerManager . . . . . 41
4.3 Struktura EventLog . . . . . 44
4.4 Klasa EventManager . . . . . 44
4.5 Przykładowe enumeracje zdarzeń . . . . . 45
4.6 Funkcja emitująca zdarzenie . . . . . 46
4.7 Klasa EventEmitter . . . . . 46
4.8 Implementacja funkcji log_event . . . . . 47
4.9 Implementacja funkcji get_event . . . . . 48
4.10 Inicjalizacja systemu plików . . . . . 55
4.11 Pobieranie stanu karty SD . . . . . 56
4.12 Definicja interfejsu ISensor . . . . . 57
4.13 Definicje typów czujników i danych . . . . . 58
4.14 Inicjalizacja czujników w klasie SensorWrapper . . . . . 61
4.15 Odczytywanie danych z czujników w klasie SensorWrapper . . . . . 62
4.16 Funkcja pobierająca czas z układu DS3231 . . . . . 63
4.17 Funkcja zbierająca dane GPS . . . . . 68
5.1 Asynchroniczne wywołanie komendy . . . . . 74

# Spis tabel


3.1 Konfiguracja kanałów pomiarowych INA3221 . . . . . 11

3.2 Parametry zastosowanego panelu fotowoltaicznego . . . . . 14

3.3 Pobór prądu modułu SX1278 . . . . . 19

4.1 Typy operacji . . . . . 36

4.2 Struktura ramki komunikacyjnej . . . . . 37

4.3 Grupy komend . . . . . 37

4.4 Komendy diagnostyczne systemu . . . . . 40

4.5 Komendy systemu zdarzeń . . . . . 49

4.6 Zawartość struktury `TelemetryRecord` . . . . . 51

4.7 Zawartość struktury `SensorDataRecord` . . . . . 51

4.8 Komendy systemu telemetrii . . . . . 54

4.9 Komendy systemu zegara . . . . . 65

4.10 Komendy systemu lokalizacji . . . . . 70